

TOPST 단기 교육 과정

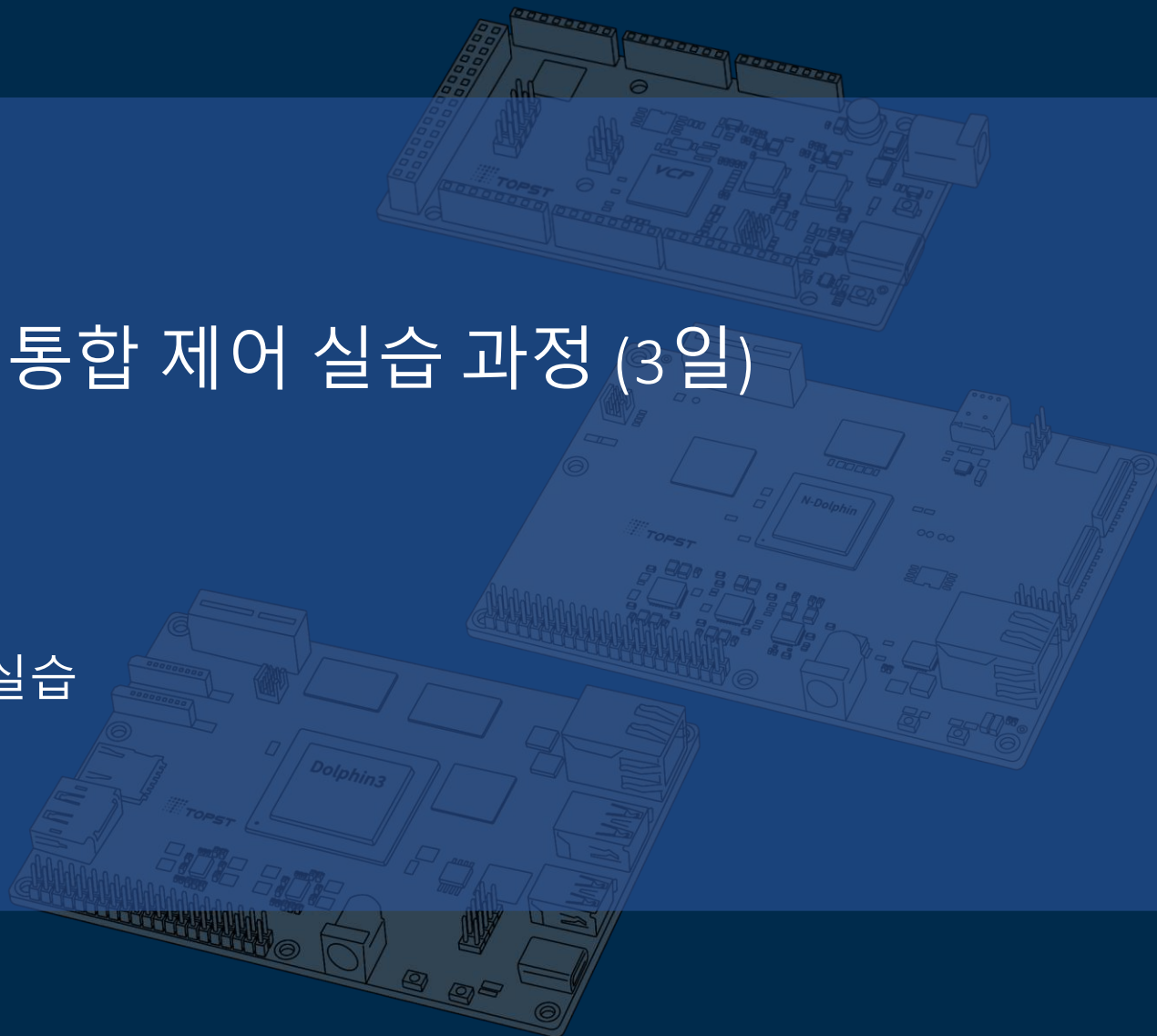
TOPST 임베디드·AI 통합 제어 실습 과정 (3일)



2일차

NPU 기반 영상 처리 실습

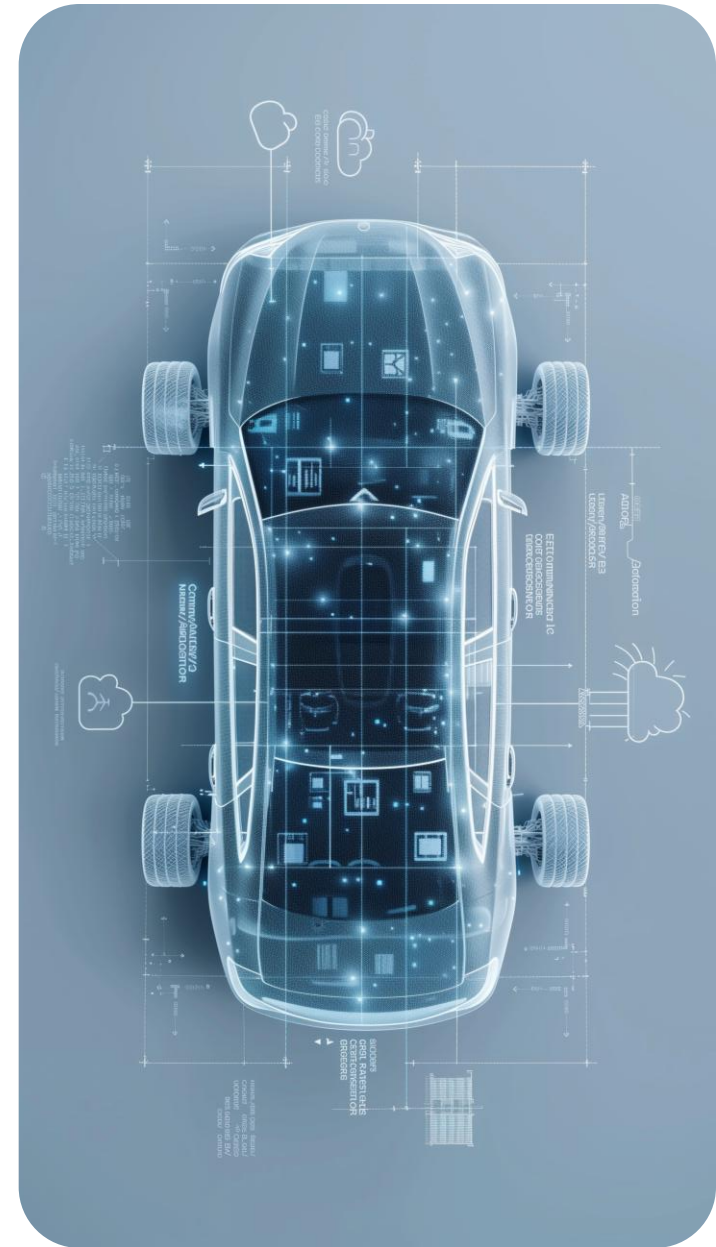
Telechips  **TOPST**



목차

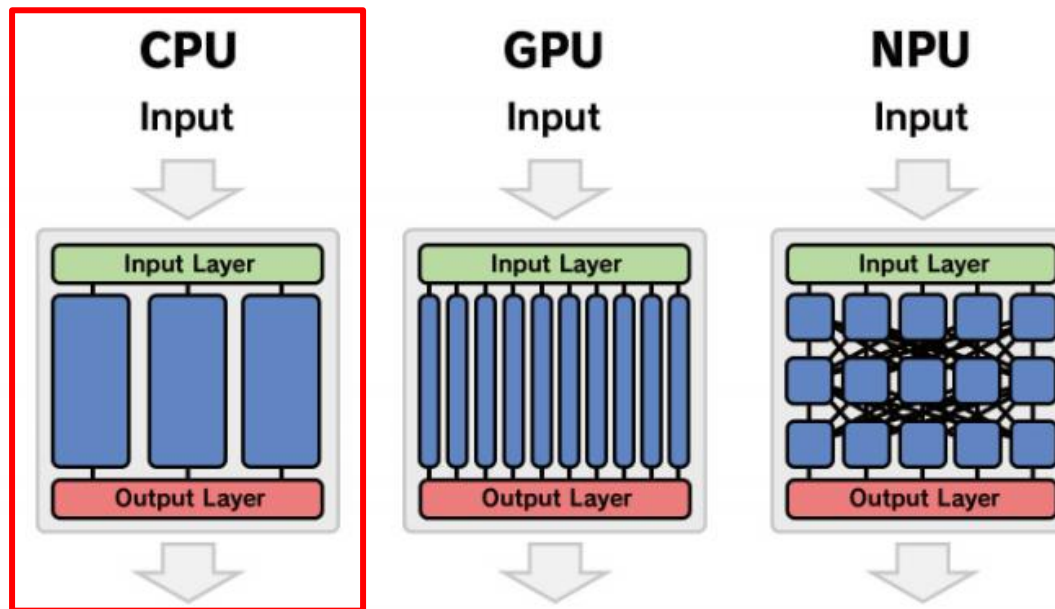
Table of Contents

TOPST AI-G 개요	01
AI-G 개발 환경 설정	02
Yolov8 모델 변환 및 최적화	03
RTPM 기반 모델 추론 실습	04
D3-G 개발 환경 설정	05



TOPST AI-G 개요

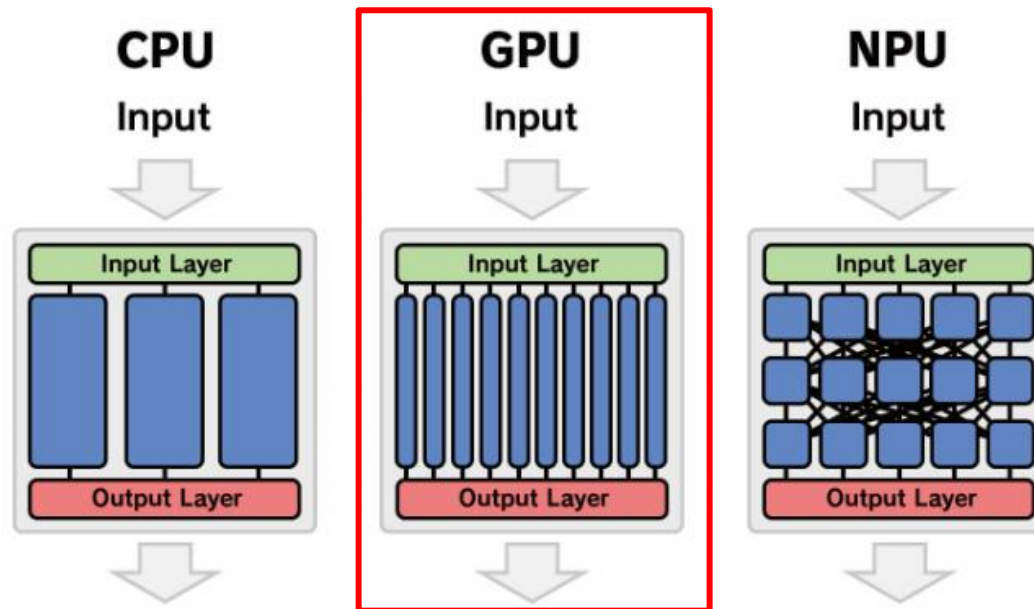
CPU vs NPU vs GPU



- 순차적 처리에 최적화된 장치로 한 번에 적은 수의 작업을 빠르게 처리
- 모델 학습 및 추론에서 병렬 처리에는 제한이 있어 대규모 연산에는 비효율적

TOPST AI-G 개요

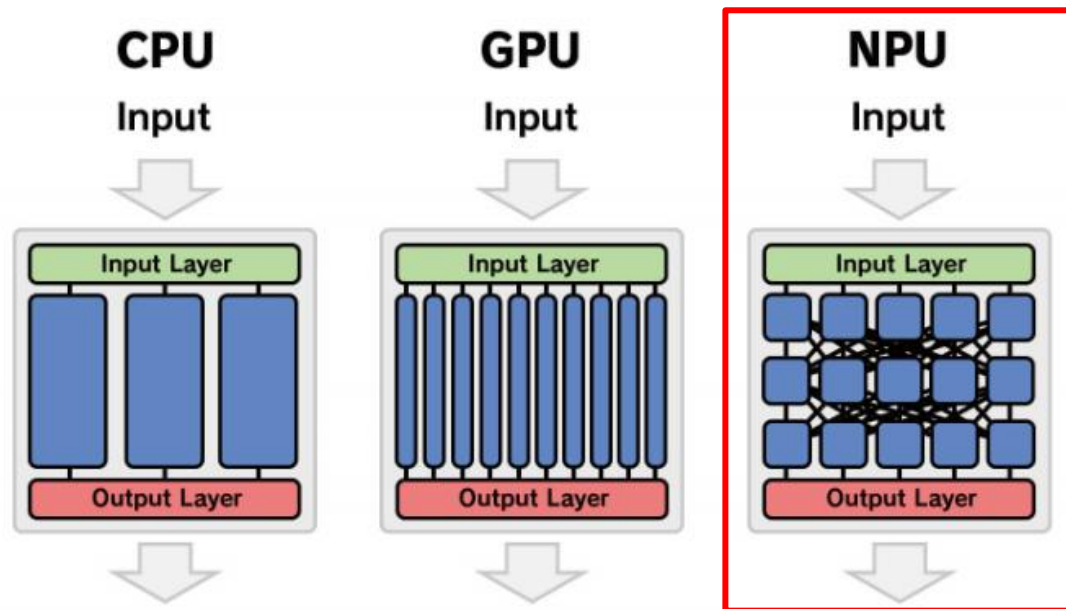
CPU vs NPU vs GPU



- 대규모 병렬 연산에 최적화된 구조, 수천 개의 작은 연산을 동시에 처리
- 복잡한 수학적 연산을 빠르게 처리할 수 있어 3D 그래픽 처리, AI 모델의 학습과 추론에 널리 사용

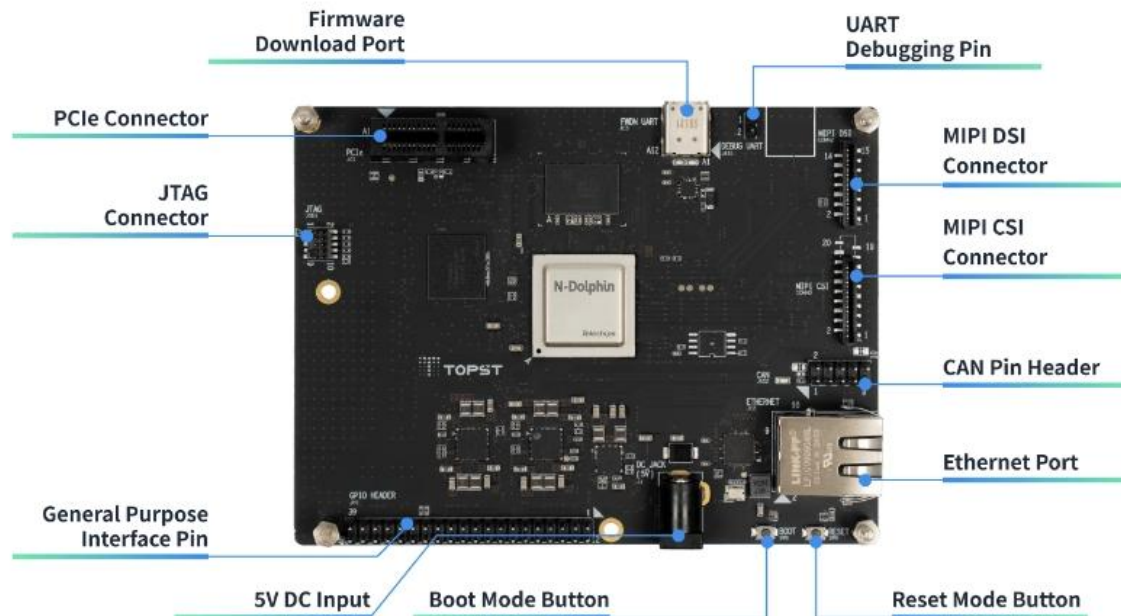
TOPST AI-G 개요

CPU vs NPU vs GPU



- AI 연산에 특화된 하드웨어로, 매트릭스 연산과 텐서 처리에 최적화
- 고속 연산, 저전력을 요구하는 임베디드 시스템에서 주로 사용
- CNN, RNN, LLM 과 같은 딥러닝 모델 처리에 매우 효율적

TOPST AI-G 개요



TOPST AI-G NPU



- 4TOPS + 4TOPS 듀얼 클러스터
- CNN과 같은 이미지 처리 특화
- 저전력, 고성능 NPU
- 동시에 2가지 모델 추론
- CPU가 내장되어 있어 stand alone으로 모델 추론 및 시각화

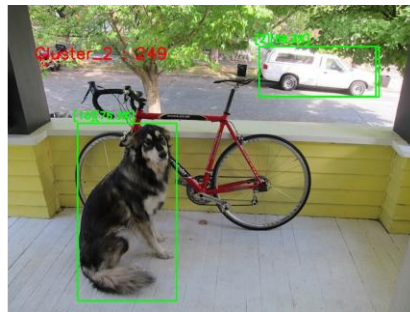
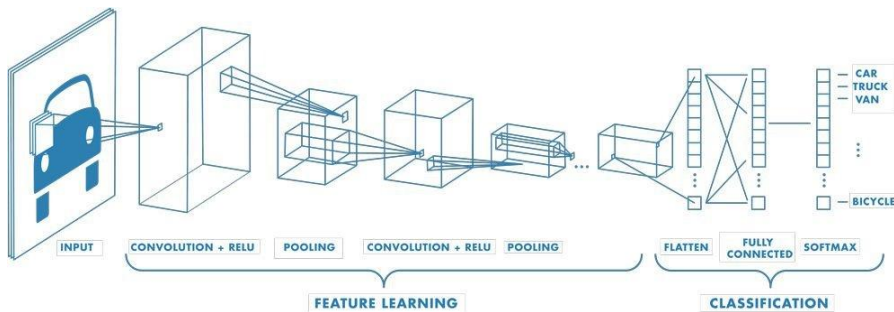
TOPST AI-G 개요

Part Name	AI-G
CPU	Arm® Cortex-A53 1.45 GHz, Quad
NPU	1.0 Ghz, 8 TOPS
Memory	2 GB 32-bit LPDDR4X
Storage	8 GB eMMC 5.1 Flash
Network	Gigabit Ethernet
Camera	MIPI CSI-2 2-lane, MIPI CSI-2 4-lane
Display	MIPI DSI-2 2-lane
Button	System reset button, Boot mode button
Others	PCIe Gen3, FWD debugger port, I2S, I2C, SPI,CAN, JTAG
Power	5V/3A
OS	Linux
Mechanical	120 mm x 90 mm

AI-G Specification

- AI-G 는 1개의 CPU와 1개의 NPU 존재
 - Cortex-A53
 - NPU : 8TOPS (4TOPS Dual)
 - 2GB RAM
 - 8GB eMMC
 - Ethernet, DSI, CSI, PCIe,

TOPST AI-G 개요



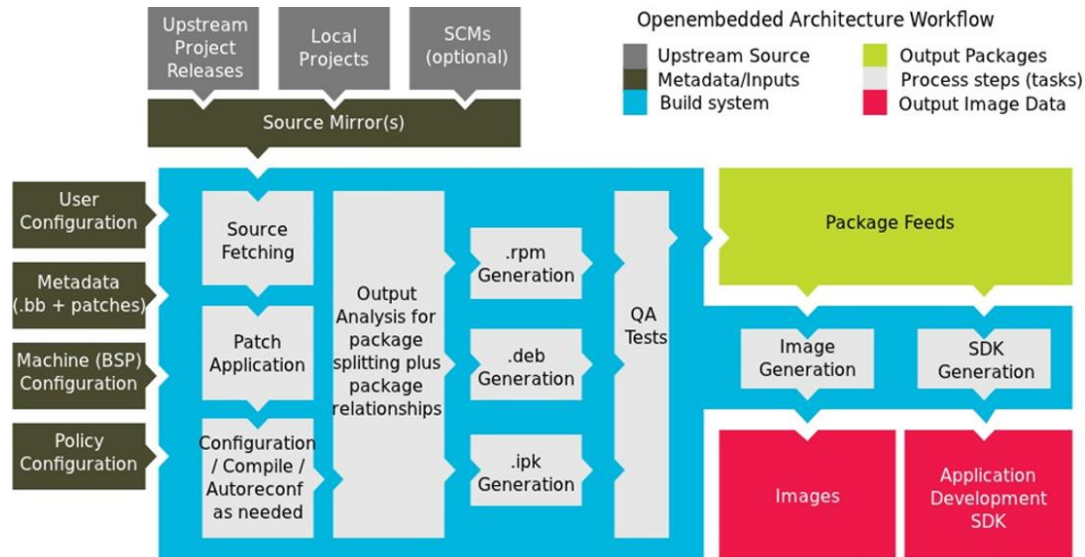
TOPST AI-G

- 영상 처리 특화 NPU
 - CNN 기반 딥러닝 네트워크 추론 특화
 - 카메라, 이미지에 대한 추론
 - 독립된 2개의 클러스터로 한 이미지에 대한 동시 추론 진행 가능
 - CPU가 내장되어 있어 단일 보드 형태로 네트워크 추론 가능

AI-G 개발 환경 설정

Yocto 란?

- Embedded Linux를 빌드하고 배포하기 위한 Framework
 - User-defined Linux platform (Kernel, boot loader, rootfs)
 - 다양한 Hardware, architecture, devices, applications
- Bitbake, Recipe 기반 빌드 시스템
- 유지보수의 편의성 제공



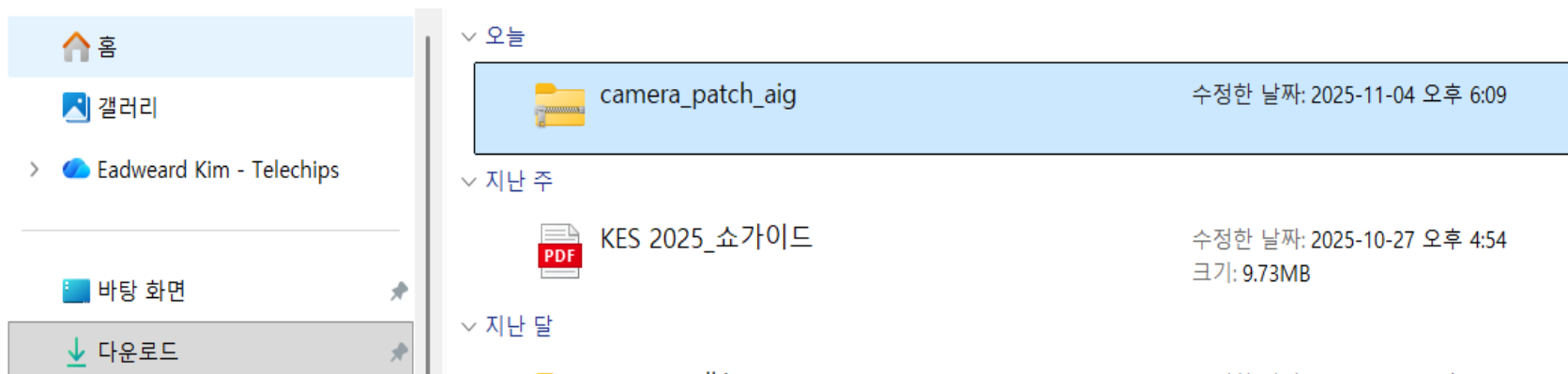
AI-G 개발 환경 설정

Yocto 란?

- 임베디드 시스템에 맞는 리눅스 커널과 시스템 파일들을 구성 및 빌드하고 이를 배포하기 위한 오픈소스 프로젝트
- **장점**
 - Target 보드의 특성에 맞는 환경 설정 가능
 - 의존성에 따른 소프트웨어와 하드웨어 구조 설정 가능
 - 변경 사항에 대해 재사용이 가능
 - 커스텀 리눅스 설치의 어려움 해결
- **단점**
 - 배우기 어려움
 - 빌드 시간이 오래 걸림
 - 빌드 에러에 대한 대처가 용이하지 않음

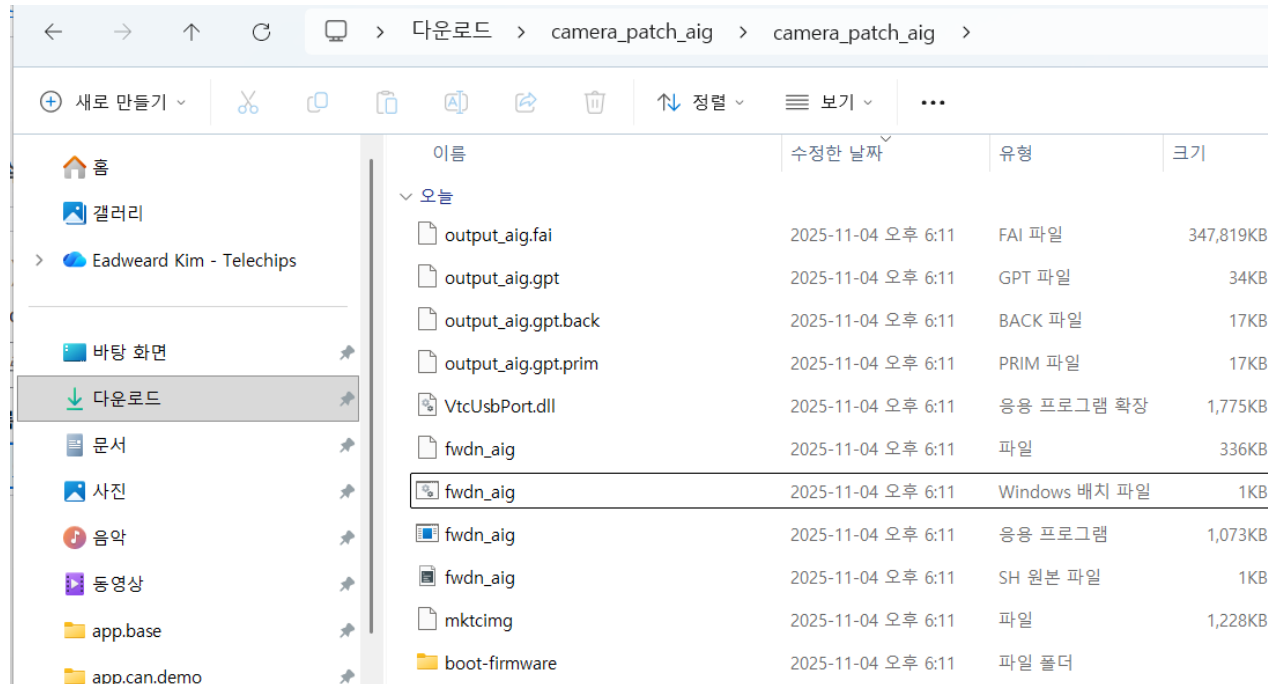
AI-G 개발 환경 설정

설치한 AI-G Yocto 파일을 다운로드 탭에서 압축 해제

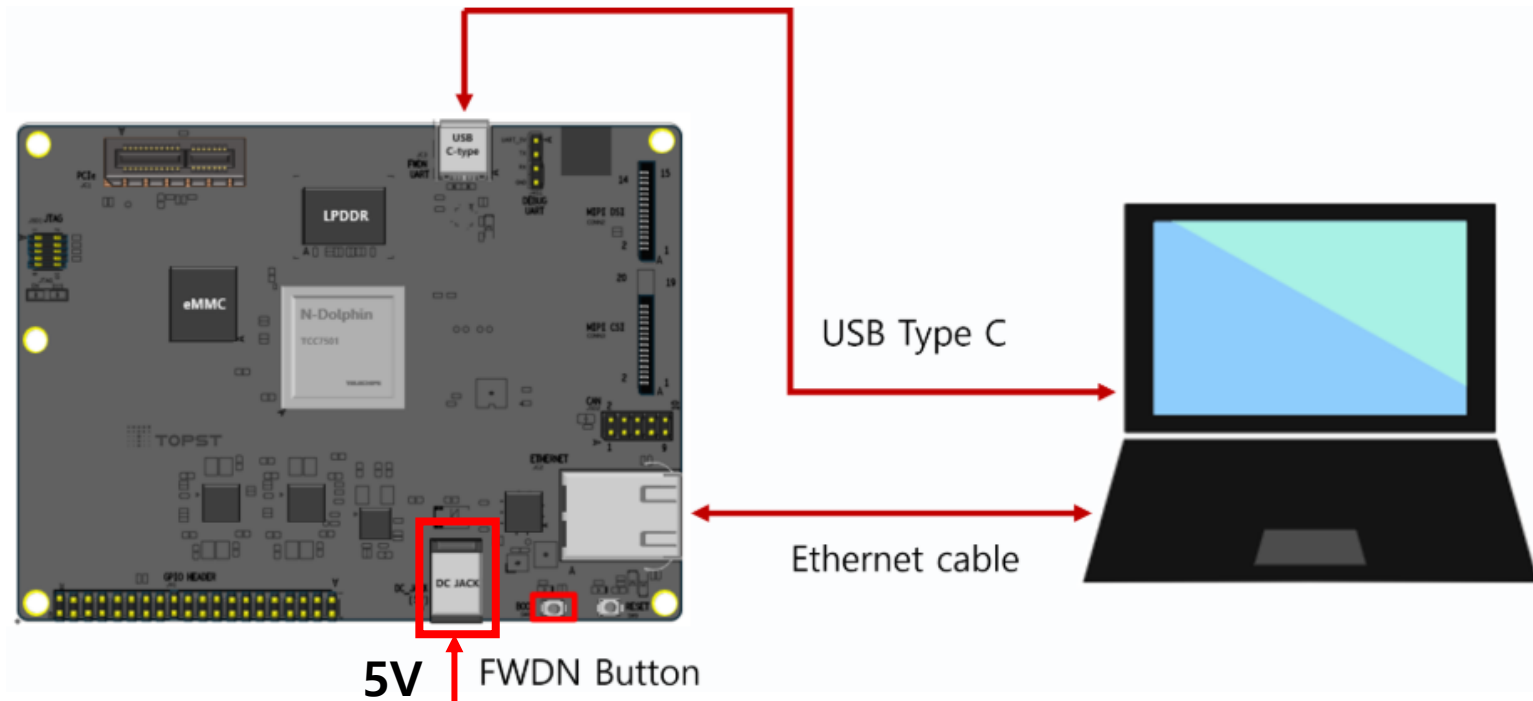


AI-G 개발 환경 설정

압축 해제한 camera_patch_aig 폴더 내부로 이동
fwdn_aig 배치파일 존재하는지 확인



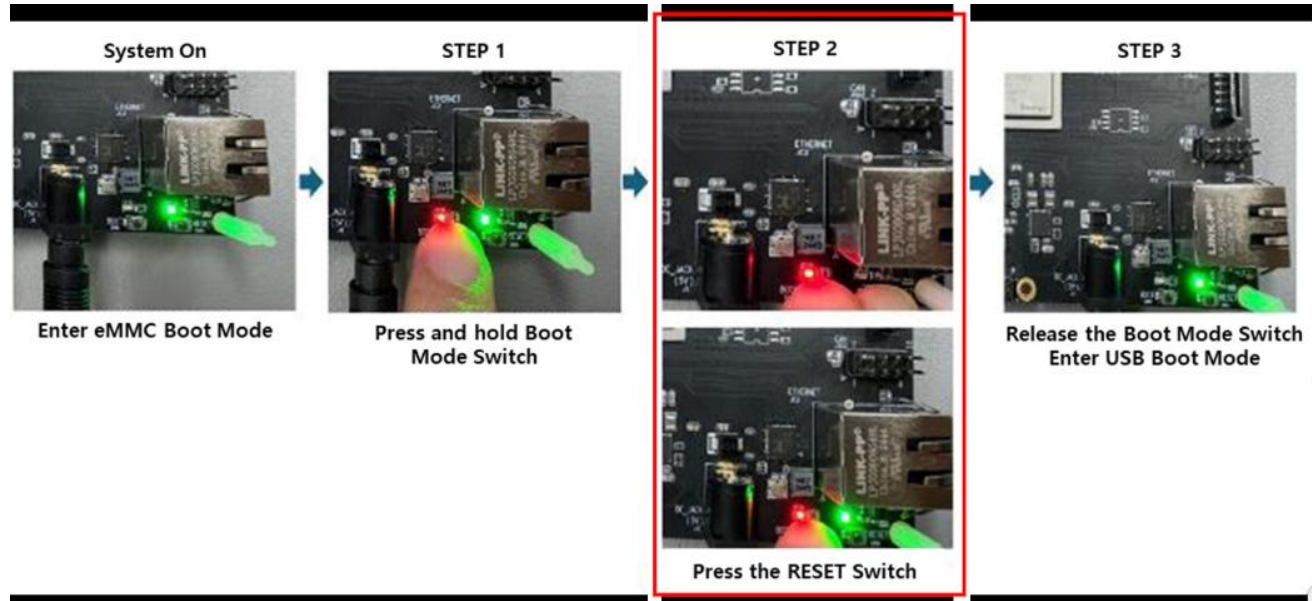
AI-G 개발 환경 설정



- **Connect AI-G Board and Host PC with USB Boot Mode**

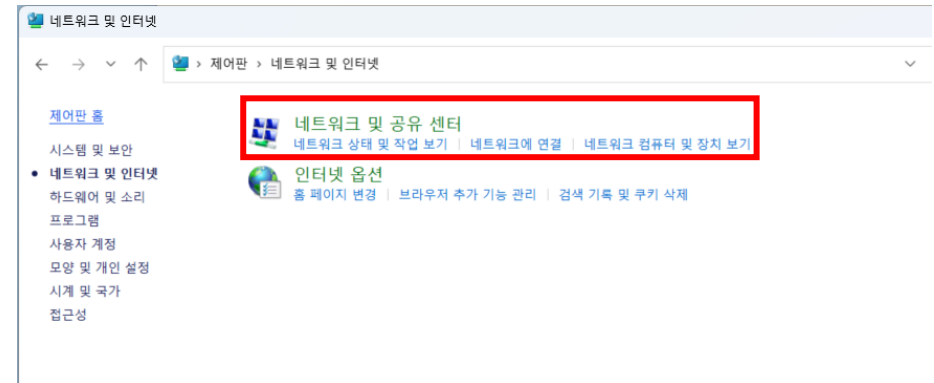
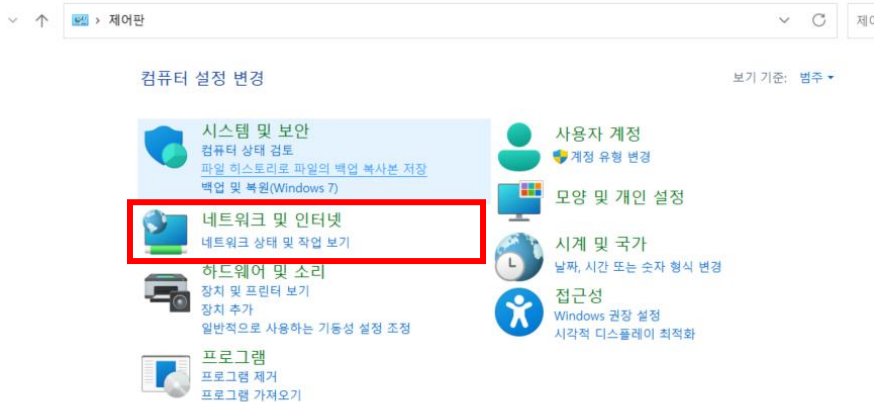
- USB Type-C 케이블을 Host PC 및 AI-G의 USB Type-C FWDN 포트에 연결
- 이더넷 케이블을 AI-G 보드의 이더넷 포트와 Host PC에 연결
- FWDN 스위치(Boot)를 누른 상태에서 전원 케이블(5V)을 AI-G 보드에 연결

AI-G 개발 환경 설정



- **Connect AI-G Board and Host PC with USB Boot Mode**
 - USB Type-C 케이블을 Host PC 및 AI-G의 USB Type-C FWDN 포트에 연결
 - 이더넷 케이블을 AI-G 보드의 이더넷 포트와 Host PC에 연결
 - FWDN 스위치(Boot)를 누른 상태에서 전원 케이블(5V)을 AI-G 보드에 연결

AI-G 개발 환경 설정



• Ethernet Set Up

- AI-G의 경우 FWDN 속도가 느리기 때문에 이더넷을 이용하여 FWDN 진행
- PC와 연결된 이더넷을 **192.168.0.8**로 설정하여 HOST PC와 AI-G가 통신할 수 있도록 설정 진행
- Host PC Network Configuration
 - 제어판 -> 네트워크 및 인터넷 -> 네트워크 및 공유 센터 -> 어댑터 설정 변경

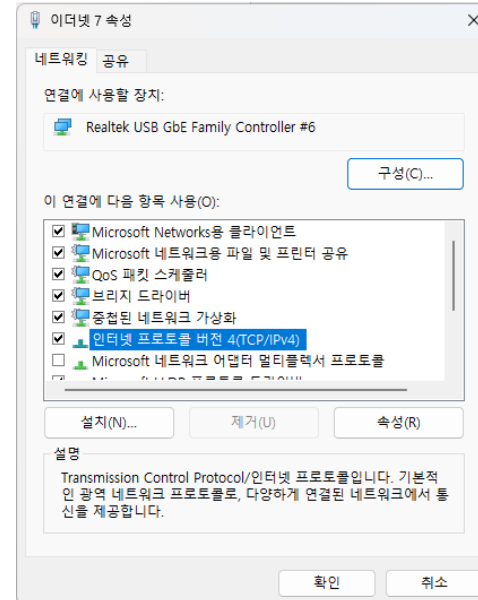
AI-G 개발 환경 설정



• Ethernet Set Up

- AI-G의 경우 FWDN 속도가 느리기 때문에 이더넷을 이용하여 FWDN 진행
- PC와 연결된 이더넷을 **192.168.0.8**로 설정하여 HOST PC와 AI-G가 통신할 수 있도록 설정 진행
- Host PC Network Configuration
 - 제어판 -> 네트워크 및 인터넷 -> 네트워크 및 공유 센터 -> 어댑터 설정 변경

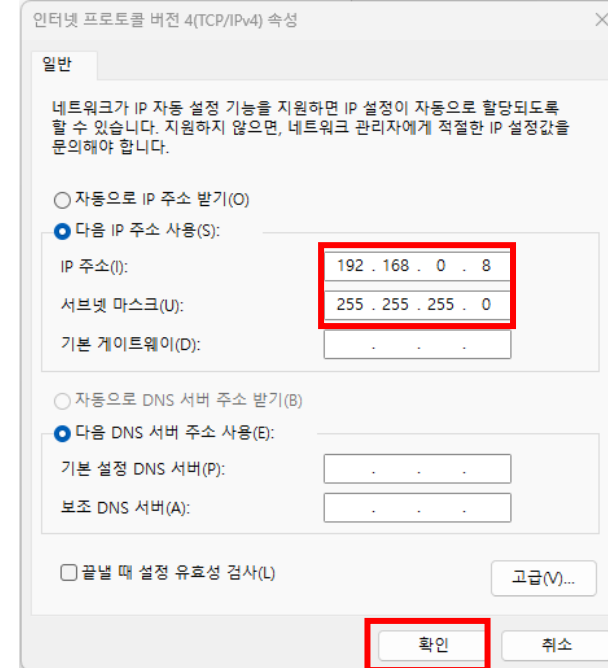
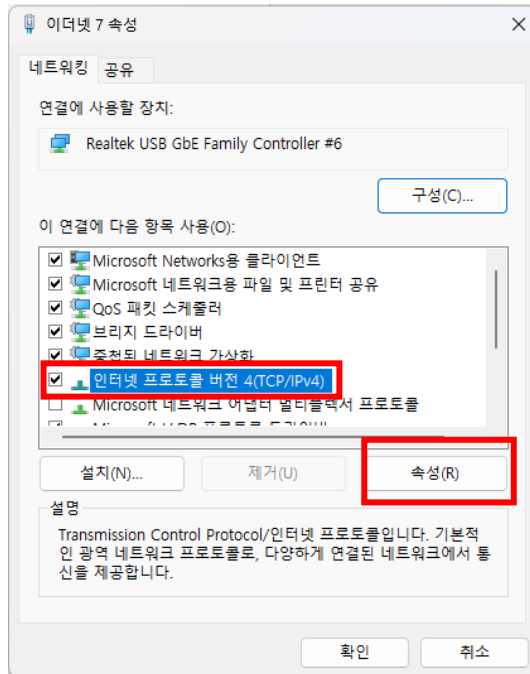
AI-G 개발 환경 설정



• Ethernet Set Up

- AI-G의 경우 FWDN 속도가 느리기 때문에 이더넷을 이용하여 FWDN 진행
- PC와 연결된 이더넷을 **192.168.0.8**로 설정하여 HOST PC와 AI-G가 통신할 수 있도록 설정 진행
- Host PC Network Configuration
 - AI-G와 연결된 이더넷 -> 오른쪽 클릭 -> 속성

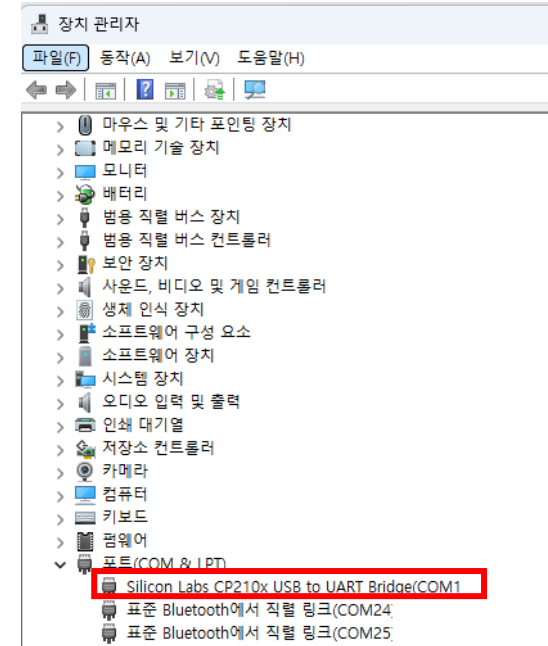
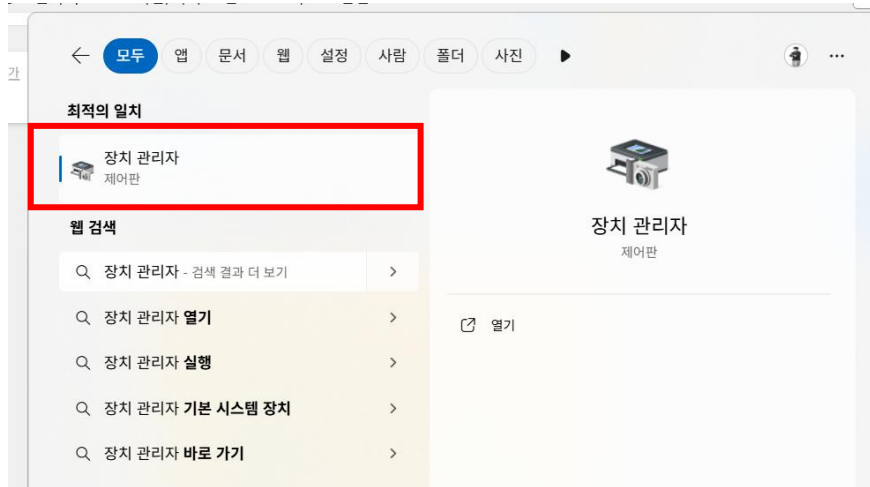
AI-G 개발 환경 설정



• Ethernet Set Up

- AI-G의 경우 FWDN 속도가 느리기 때문에 이더넷을 이용하여 FWDN 진행
- PC와 연결된 이더넷을 **192.168.0.8**로 설정하여 HOST PC와 AI-G가 통신할 수 있도록 설정 진행
- Host PC Network Configuration
 - 인터넷 프로토콜 버전 4(TCP/IPv4) -> 속성
 - 다음 주소 사용 -> IP : 192.168.0.8 / NETMASK : 255.255.255.0 입력 -> 확인

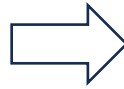
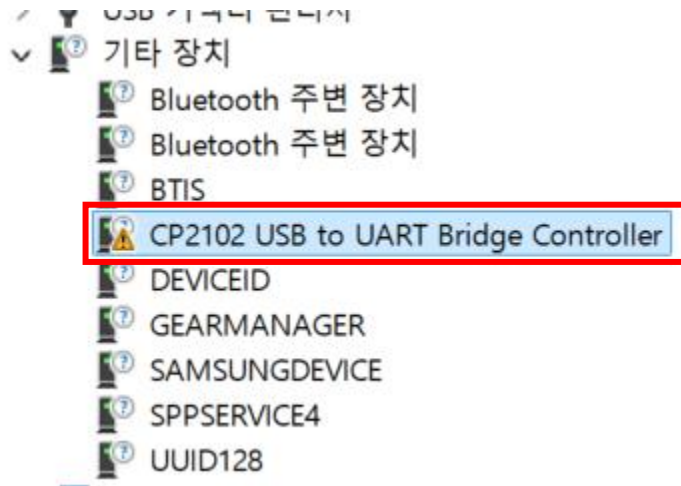
AI-G 개발 환경 설정



• FWDN (AI-G Yocto Image)

- 검색창에 장치 관리자 검색 후 실행
- 포트 탭에서 연결된 장치(CP210x USB to UART) 시리얼 번호 (COM xx 확인)

AI-G 개발 환경 설정



바 찾아보기

검색:

₩OneDrive - Telechips₩문서

가능한 드라이버 목록에서 직접 선택
기스와 호환되는 사용 가능한 드라이버 및 디바이스
표시됩니다.

폴더 찾아보기

하드웨어에 대한 드라이버가 들어 있는 폴더를 선택하십시오.

32-mercedes-benz-gls-580-2020
> ADR033-Adept_Robotic_Car_Kit_for_Pico-20231216
> archive
> B257212
> B257212 (3)
> camera_patch_aig
> camera_patch_aig (1)
✓ CP210x_Universal_Windows_Driver
 arm
 arm64
 x64
 x86
 x86_64

• 장치가 보이지 않을 경우

- 기타 장치에서 CP2102 우클릭 -> 드라이버 업데이트 -> 내 컴퓨터에서 드라이버 찾아보기
- 찾아보기 -> 이전에 설치한 CP210x 드라이버 폴더 선택 후 확인 -> 다음
- 장치 인식 확인

AI-G 개발 환경 설정

```
C:\WINDOWS\system32\cmd.exe
TOPST AI-G FWDN Batch File

Find Port.
Silicon Labs CP210x USB to UART Bridge(COM7)
표준 Bluetooth에서 직렬 링크(COM24)
Bluetooth Device (RFCOMM Protocol TDI)
표준 Bluetooth에서 직렬 링크(COM25)

Input USB Port Number :
```



```
TOPST AI-G FWDN Batch File

Find Port.
Silicon Labs CP210x USB to UART Bridge(COM44)

Input USB Port Number : 44

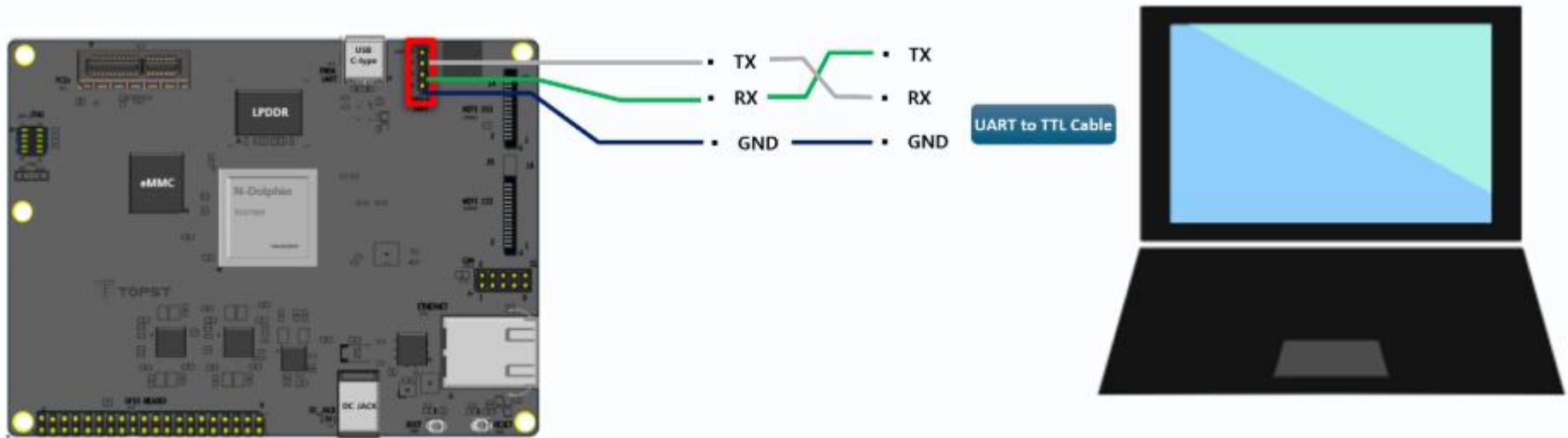
Step 1. Connect between Host PC and TOPST AI-G
[FWDNLogger::PrintCurTime:100] 25/06/12-16:11:36
[main:24] FWDN N-Dolphin v1.0.1 - 2024.9.24 16:18:11
[main:92] Start FWDN
[FWDN_ND::ImageSend:442] Complete to send MCERT image
[FWDN_ND::ImageSend:442] Complete to send HSM image
[FWDN_ND::ImageSend:442] Complete to send FWDN_BL1 image
[FWDN_ND::ImageSend:442] Complete to send FWDN_DRAM_PARAMETER image
[FWDN_ND::ImageSend:442] Complete to send FWDN_BL2 image
[FWDN_ND::ImageSend:442] Complete to send FWDN_BL3 image
[FWDN_ND::InfoDevice:123] ##### Device Info #####
[PrintIP:527] ip : 192.168.0.100
[PrintIP:527] mask : 255.255.255.0
[PrintIP:527] gateway : 0.0.0.0
[FWDN_ND::InfoDevice:132] port : 8080
[FWDN_ND::InfoDevice:135]
mmc0_user: 7818182656 byte block_size: 512 byte
mmc0_boot0: 4194304 byte block_size: 512 byte
mmc0_boot1: 4194304 byte block_size: 512 byte
snor1: 0 byte
snor2: 0 byte
fw_version: tcc750x_v0.0.111
fw_build_id: g90e2b777
fw_build_date: 2024-09-24-17:11:42+0900

----- Firmware Information -----
VERSION : tcc750x_v0.0.111
BUILD ID : g90e2b777
BUILD DATE : 2024-09-24-17:11:42+0900
```

• FWDN 실행

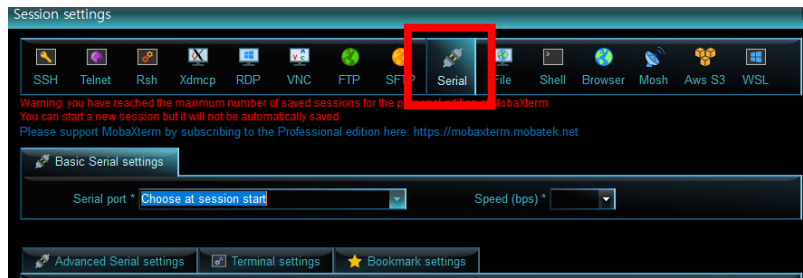
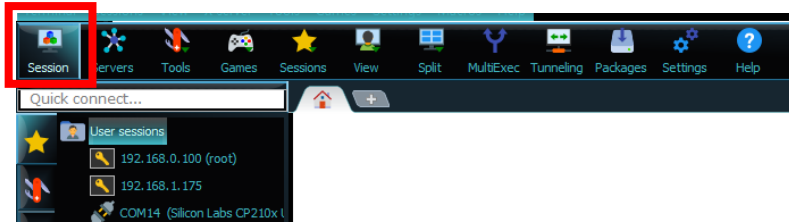
- 이전에 설치한 camera_patch_aig 파일 내부로 이동
- fwdn_aig (windows 배치파일) 실행
- USB 포트 넘버에 확인한 시리얼 장치(CP210x) 번호 입력 – ex) 7 입력 (COM7)

AI-G 개발 환경 설정



- **Connect AI-G Board and Host PC with UART**
 - AI-G 보드와 PC 간 USB-to-TTL cable을 이용하여 연결
 - 검정 케이블은 UART 핀의 **GND**에 연결
 - 흰 케이블은 UART 핀의 **TX** 핀에 연결 & 녹색 케이블은 UART 핀의 **RX** 핀에 연결

AI-G 개발 환경 설정



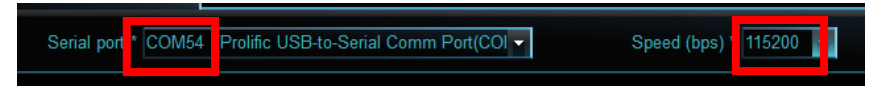
• Connect AI-G Board and Host PC with UAR

- 보드 접속
 - MobaXterm 실행
 - 장치 관리자를 통해 port 확인
 - ✓ 포트(COM & LPT)
 - Prolific USB-to-Serial Comm Port(COM36)
 - Silicon Labs CP210x USB to UART Bridge(COM4)
 - Session -> Serial 탭 클릭
 - Serial port 탭에서 장치관리자를 통해 확인한 포트 선택
 - Speed '115200'으로 설정

AI-G 개발 환경 설정

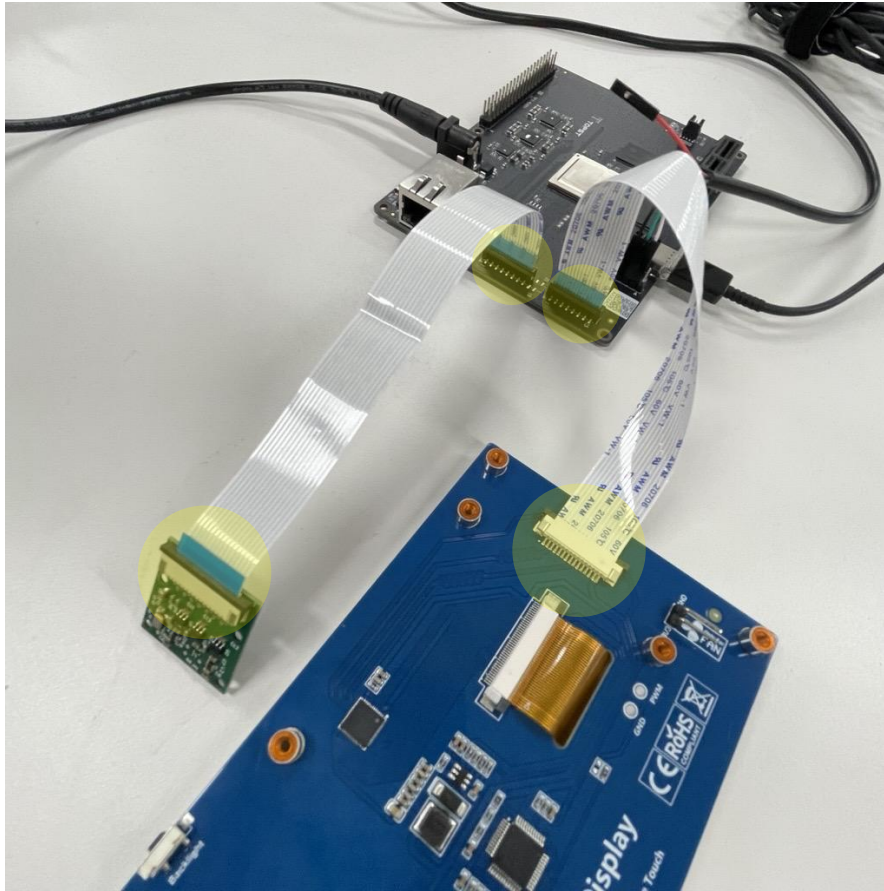


- Serial port에서 연결된 포트 설정
- Speed 115200 설정



- Advanced Serial settings 탭 선택
- Flow control에서 None 선택
- OK 눌러 보드 접속

AI-G 개발 환경 설정

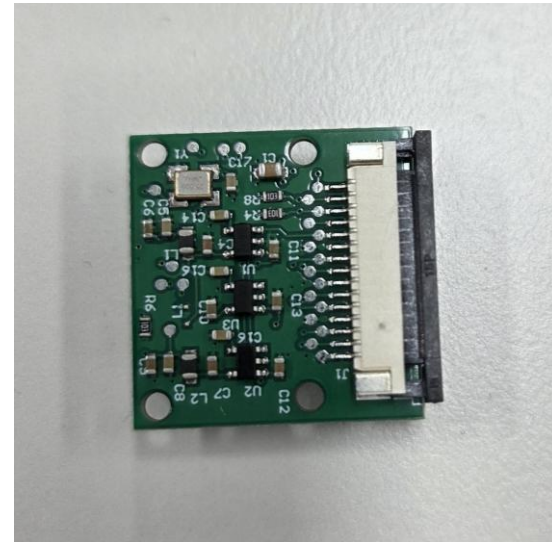
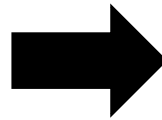


MIPI 장치간 연결 주의사항

1. 반드시 화면과 같이 연결
 - FFC 케이블 부착 방향 주의
2. 디바이스 연결 후에 AI-G 전원 연결

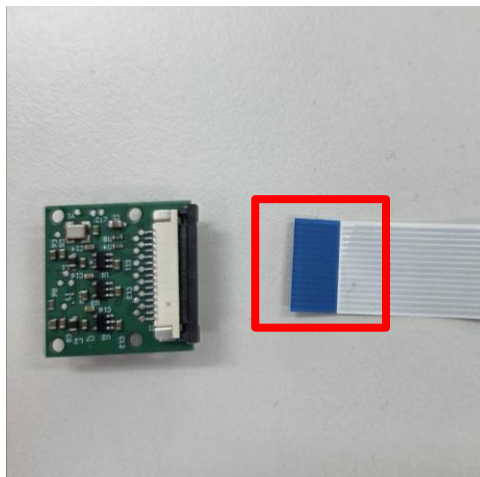
AI-G 개발 환경 설정

1. 고정 핀 해제

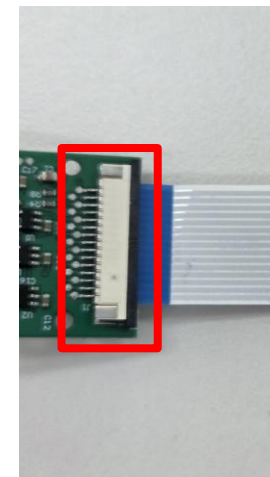
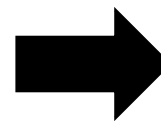
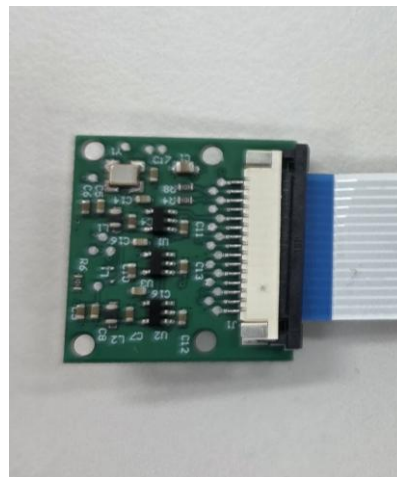
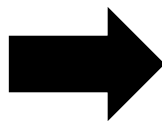


AI-G 개발 환경 설정

2. 케이블 연결



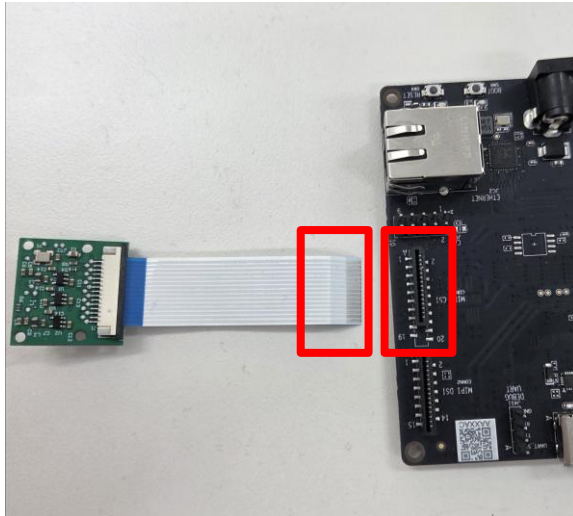
케이블 방향 확인!



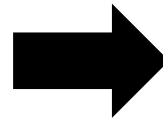
핀 고정

AI-G 개발 환경 설정

3. 카메라와 보드 연결



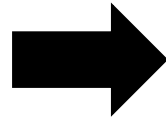
케이블 방향 확인!



카메라와 보드 연결

AI-G 개발 환경 설정

1. 고정 핀 해제

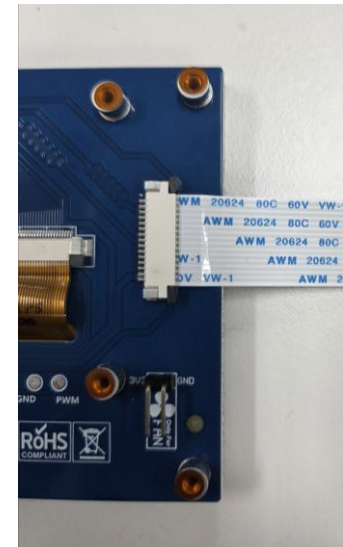
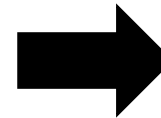
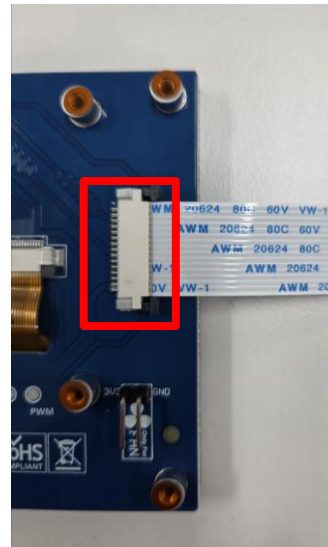
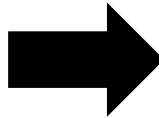


AI-G 개발 환경 설정

2. 케이블 연결



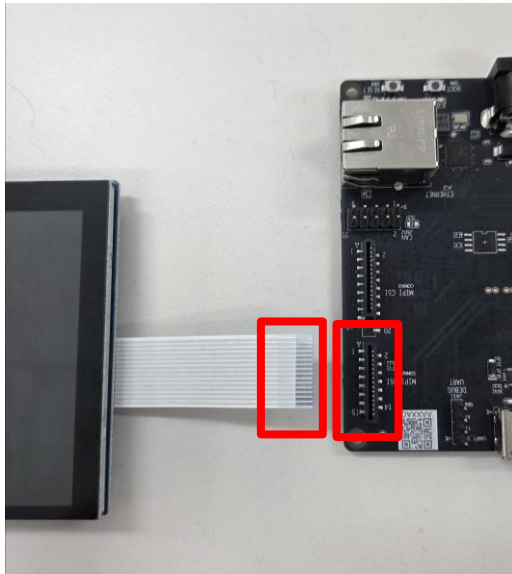
케이블 방향 확인!



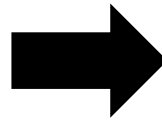
핀 고정

AI-G 개발 환경 설정

3. 보드 연결



케이블 방향 및 MIPI 포트 위치 확인!



보드와 연결

AI-G 개발 환경 설정

```
eth0: gmac@13000000
blkread: mmc 0 is current device
Hit any key to stop autoboot: 0
[tcc_fb_dm]: version : 1.4.0 20230824
Video      = tcc_fb_dm0 active
Non-secure boot (secure boot flag is clear)
## Booting Android Image at 0x24000000 ...
Kernel load addr 0x20000000 size 16517 KiB
Kernel command line: quiet fsck.repair=yes console=ttyAMA2,115200n8
## Flattened Device Tree blob at 28000000
   Booting using the fdt blob at 0x28000000
   Loading Kernel Image
ERROR: reserving fdt memory region failed (addr=20000000 size=1000000 flags=4)
   Using Device Tree in place at 0000000028000000, end 0000000028010fff
   DTBEE OS may not be stored in storage. res: 0xffffffffffffffff

Starting kernel ...

[ 1.207999] telechips-pcie 12000000.pcie: invalid resource
[ 1.334582] telechips-pcie 12000000.pcie: timeout waiting for link up
[ 1.592471] debugfs: Directory '10300000.dma' with parent 'dmaengine' already present!
[ 1.601609] debugfs: Directory '10320000.dma' with parent 'dmaengine' already present!
[ 1.610745] debugfs: Directory '10330000.dma' with parent 'dmaengine' already present!
[ 1.619885] debugfs: Directory '10340000.dma' with parent 'dmaengine' already present!
[ 1.629051] debugfs: Directory '10350000.dma' with parent 'dmaengine' already present!
[ 4.990360] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0
[ 4.999920] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0
[ 5.009469] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0
[ 5.019004] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0
[ 5.028523] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0
[ 5.038059] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0
[ 5.047577] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0
[ 5.057141] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0
[ 5.057155] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0
[ 5.076165] tcc-dewarp videoinput0: [ERROR][DEWARP] tcc_dewarp_ioctl_enum_framesizes - index(1) is not 0

Telechips Baseline (Poky/meta-telechips/meta-core) 4.0.17 ai-g-topst ttyAMA2
ai-g-topst login: |
```

```
root@ai-g-topst:~# _
```

- 보드 실행
 - Reset 버튼을 누르거나 전원 재인가
 - 디버깅 창 확인
 - ai-g-topst login : root / password : root 입력

AI-G 개발 환경 설정

TOPST AI-G 모델 추론

```
$ tcnnapp -p /dev/video2
```

```
root@ai-g-topst:~# tcnnapp -n lenet_quantized/ -p /dev/video2

=====
==
==
==
==
==
==
==
==
==
==
=====

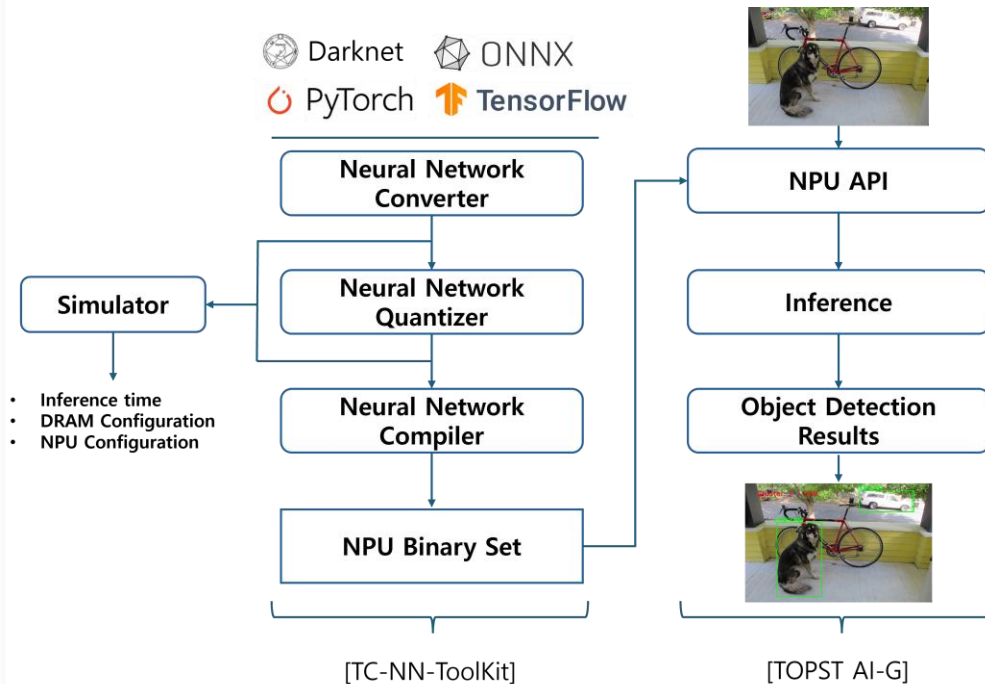
[Info] reading framebuffer resolution from fb0
##### finall Width = 800 and Height=480
-----Parameter Info-----

[Network1]Network Path      : lenet_quantized/
[Network2]Network Path      : /usr/share/mobilenetv2_10_quantized/

[Common]Input Mode          : camera
[Common]Output Mode         : display
[Common]Input Size          : 1280 x 720
[Common]Output Size         : 800 x 480
[Common]Input Path          : /dev/video2
[Common]Output Path         : /dev/overlay
[Common]Output Position X   : 0
[Common]Output Position Y   : 0

[Debug]Debug Mode           : off
[Debug]NPU Debug Mode       : off
[Debug]NPU Running Mode     : AsyncMode
Parameter End
```

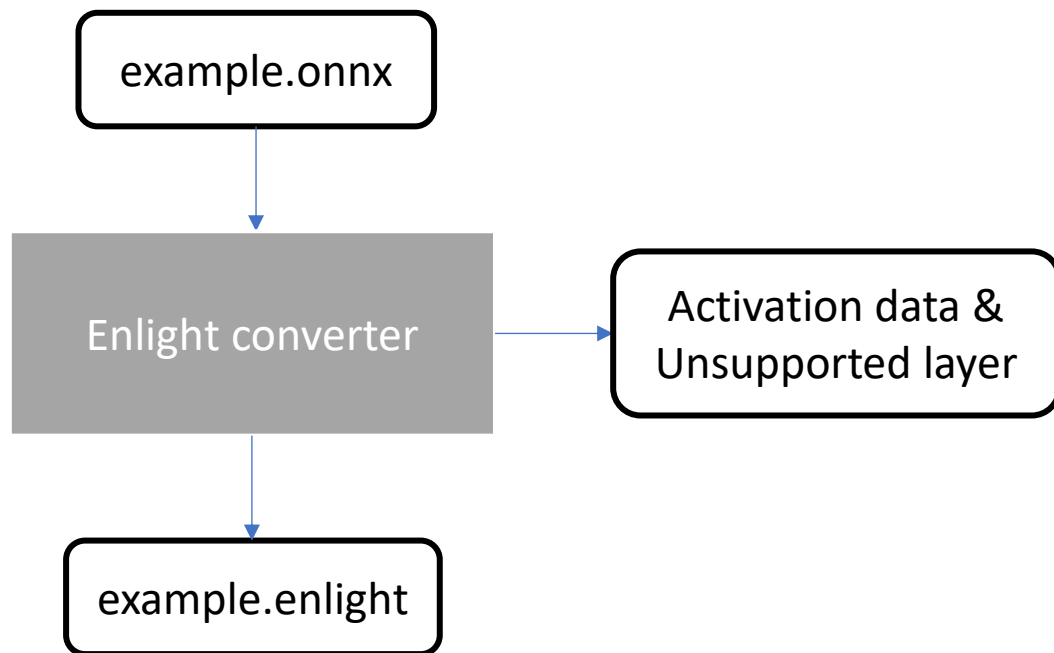
Yolov8 모델 변환 및 최적화



1. tc-nn-toolkit 소개

- NPU에서 구동하기 위한 모델 변환 툴
- 4개의 AI 프레임 워크 모델 변환 지원
- Convert, quantize, compile 총 3단계로 구성
- Simulator를 통해 양자화 전-후 모델 평가
- float32 모델을 int8 모델로 최적화

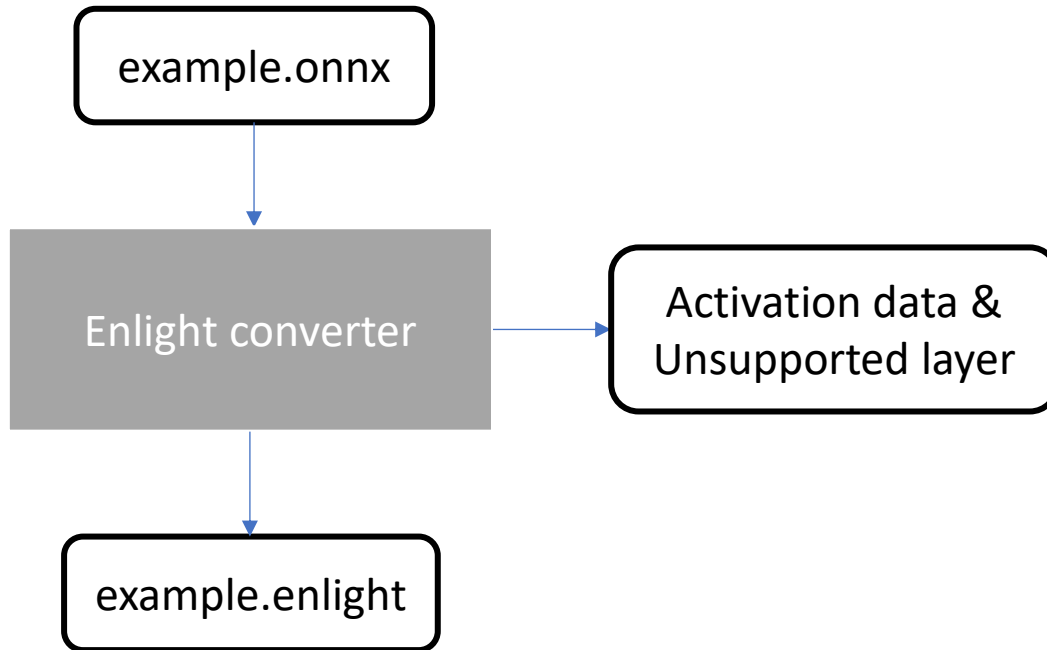
Yolov8 모델 변환 및 최적화



1. converter 이해

- Pytorch, TensorFlow, Darknet, ONNX와 같은 프레임 워크에서 학습된 모델을 변환
- TOPST AI-G NPU에서 추론을 위한 최적화된 포맷으로 변환
- 변환하는 모델의 지원되지 않는 연산자 확인
- 양자화를 위한 활성화 데이터 통계 수집

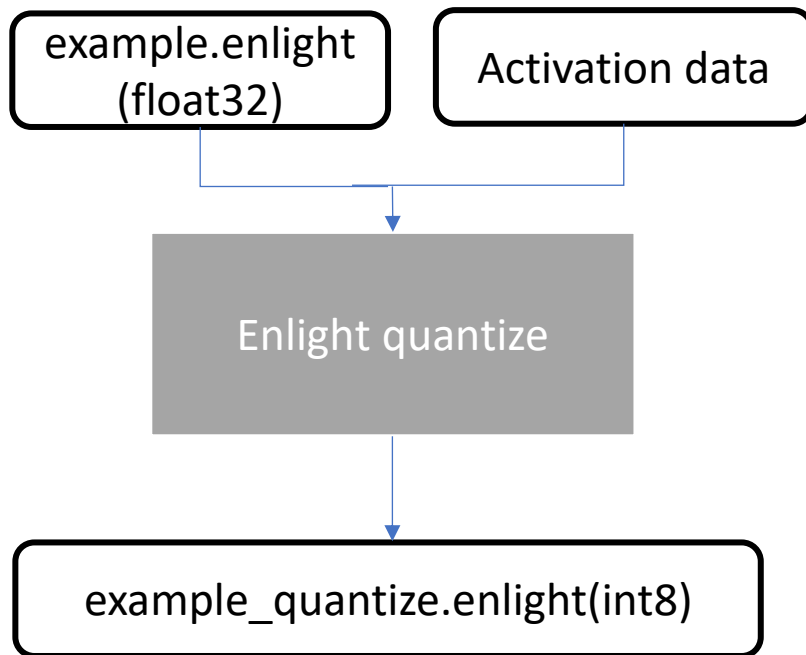
Yolov8 모델 변환 및 최적화



1. converter 이해

- 명령어 사용 방식
- Python `convert.py --{사용 옵션}`
- 주요 옵션
 - type - 후처리 타입
 - enable-track - 활성화 통계 수집 여부
 - Enable-letterbox - letterbox 활성화 여부
 - dataset - 활성화 통계 수집에 사용할 데이터셋
 - num-class - 클래스 수
- 이외에도 다수 옵션 존재

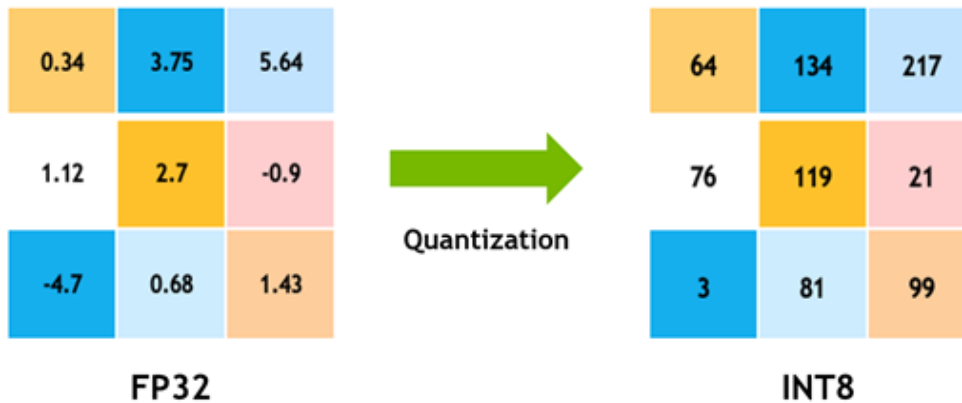
Yolov8 모델 변환 및 최적화



2. quantize 이해

- 부동소수점 연산을 정수 연산으로 변환
- Float32 모델을 int8 모델로 변환하여 모델의 크기를 줄이고 속도 향상
- 가중치와 활성화를 8bit 정수로 변환
- 활성화 데이터 통계를 통해 최적의 양자화 비율을 계산하여 적용

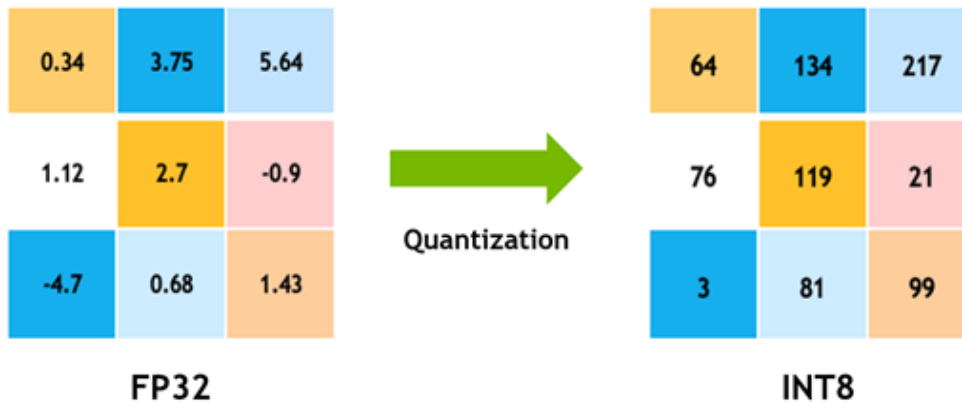
Yolov8 모델 변환 및 최적화



2. quantize 이해

- FP32 : 32비트 부동 소수점 형식.
 - 높은 정밀도를 제공
 - 메모리 공간을 다수 차지
 - 연산이 느림
- INT8 : 8비트 정수점 형식
 - 저장 공간과 연산속도에서 유리
 - 정밀도가 낮아질 수 있음
 - 저전력을 요구하는 환경에서 효율적 처리

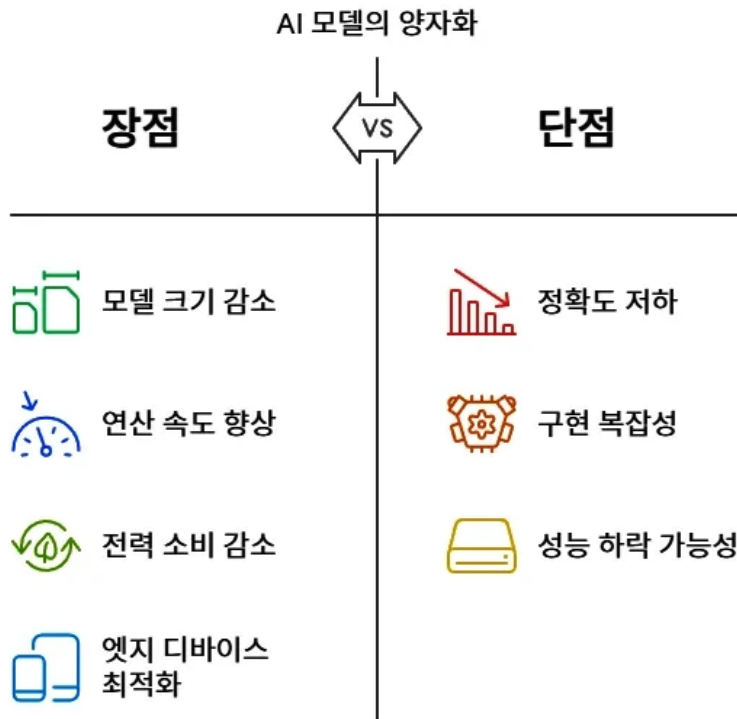
Yolov8 모델 변환 및 최적화



2. quantize 이해

- 양자화란?
 - 정수화를 통해 모델의 가중치, 활성화 값등을 정수로 변환하는 작업
- 가중치 양자화
 - 모델의 가중치 매개변수를 정수 변환
 - 양자화 스케일을 사용해 정수 변환
- 활성화 양자화
 - 신경망의 각 층의 출력 값을 정수 변환
 - 입력 데이터 범위와 스케일을 고려해 양자화된 값으로 변환

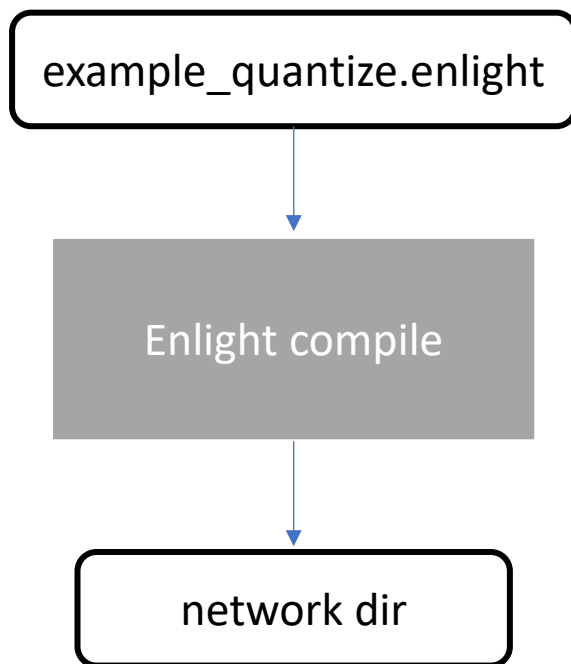
Yolov8 모델 변환 및 최적화



2. quantize 이해

- 장점
 - 모델 크기 감소 : 모델의 크기가 약 4배 줄어듦
 - 연산 속도 향상
 - 메모리 사용 절감
- 단점
 - 정밀도를 낮추기 때문에 정확도 손실

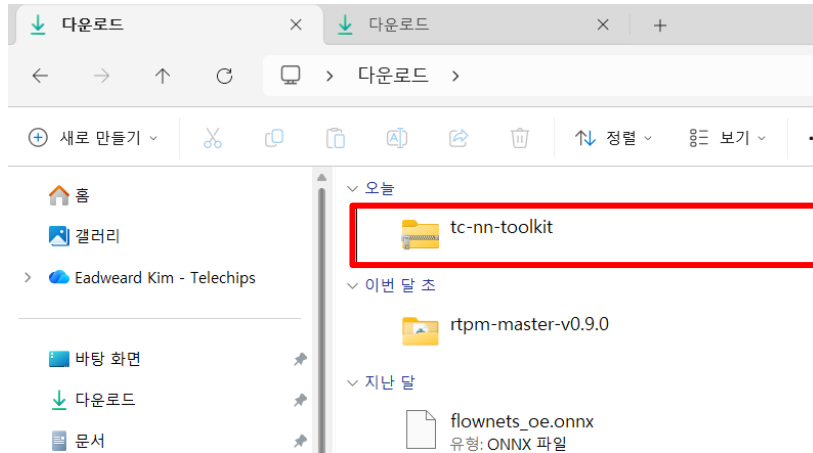
Yolov8 모델 변환 및 최적화



3. compile 이해

- 양자화된 신경망 모델을 NPU 명령어와 네트워크 파라미터로 변환하여 NPU에서 실행 가능한 형식으로 컴파일
- NPU 명령어 파일과 양자화된 가중치 등 생성
- 후처리 함수를 위한 소스 생성
- 이후 CPU에서 추론된 텐서에 대한 후처리 연산 가능

Yolov8 모델 변환 및 최적화



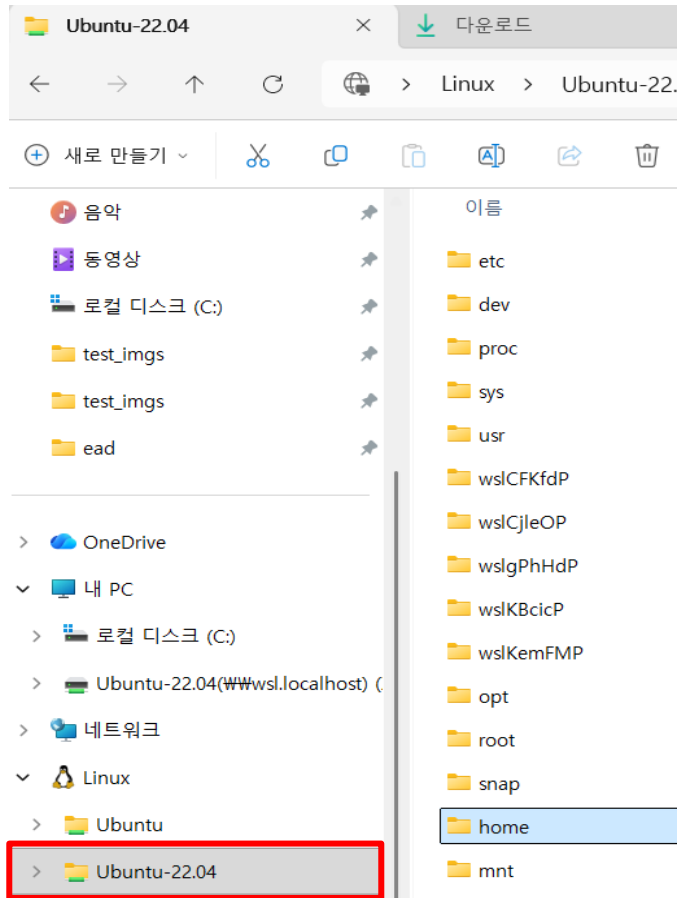
사전 설치된 tc-nn-toolkit 압축 해제
(파일 탐색기 - 다운로드)

tc-nn-toolkit 폴더를 wsl 환경으로 이동하기
위해 복사

파일 탐색기에서
Linux-> Ubuntu-22.04탭 선택

Home -> {user} 선택 후 복사한 후 붙여넣기

Yolov8 모델 변환 및 최적화



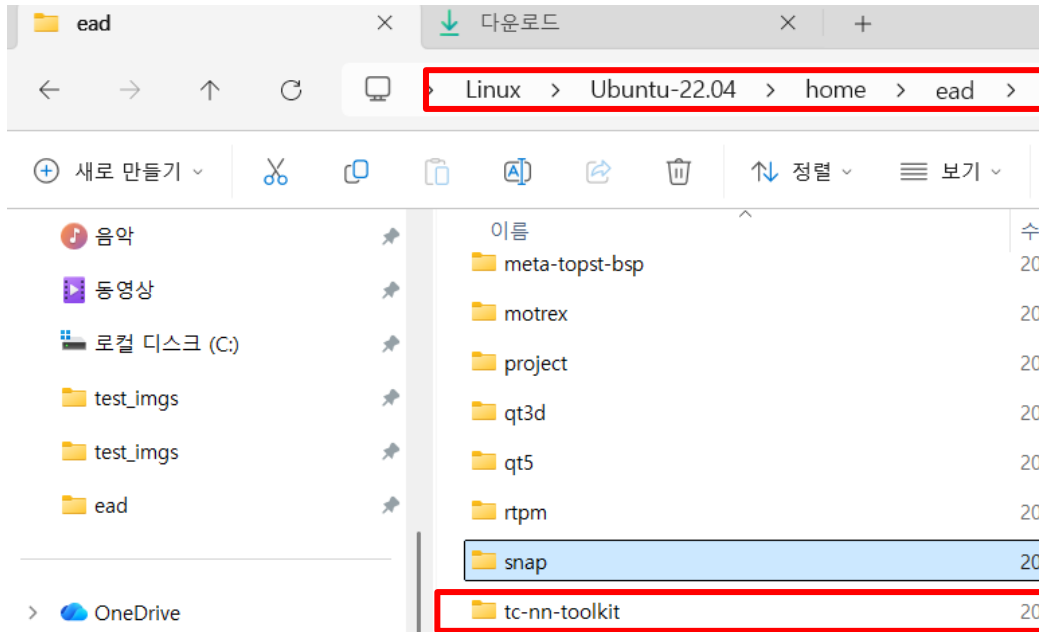
사전 설치된 tc-nn-toolkit 압축 해제
(파일 탐색기 - 다운로드)

tc-nn-toolkit 폴더를 wsl 환경으로 이동하기
위해 복사

파일 탐색기에서
Linux-> Ubuntu-22.04탭 선택

Home -> {user} 선택 후 복사한 후 붙여넣기

Yolov8 모델 변환 및 최적화



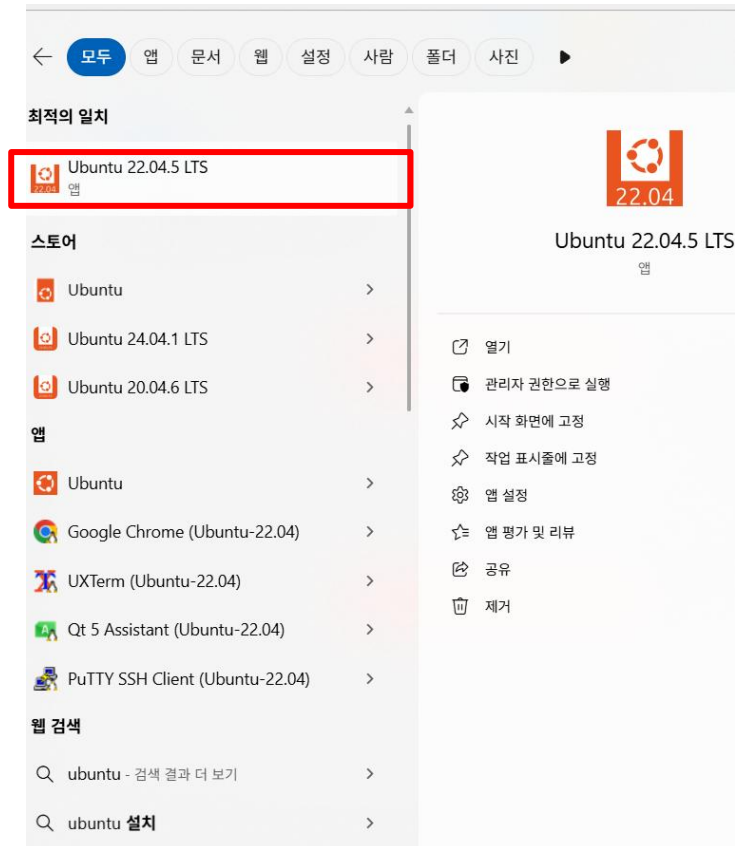
사전 설치된 tc-nn-toolkit 압축 해제
(파일 탐색기 - 다운로드)

tc-nn-toolkit 폴더를 wsl 환경으로 이동하기
위해 복사

파일 탐색기에서
Linux-> Ubuntu-22.04탭 선택

Home -> {user} 선택 후 복사한 후 붙여넣기

Yolov8 모델 변환 및 최적화



tc-nn-toolkit 환경 설정

파일 탐색기에서 ubuntu 실행

tc-nn-toolkit 디렉터리로 이동

```
$ cd tc-nn-toolkit
```

```
$ ls
```

```
head@TC-U-20-0546:~/tc-nn-toolkit/tc-nn-toolkit$ ls
build_network                               requirements_tflite.txt
custom_evaluation_enlight_classification.py  requirements.txt
custom_evaluation_enlight_object_detection.py
downloads                                  run_example_mb2_ssd_onnx.sh
enlightSDK                                run_example_mobilenetv2_onnx.sh
enlight_viewer                             run_example_pytorch_yolo_onnx.sh
inference_enlight_classification.py          run_example_ssd_mobilenetv1_tflite.sh
inference_enlight_object_detection.py        run_example_yolov4_cfg.sh
inference_mb2_ssd_onnx_onnx.py              run_toolkit_ssd_lite300.sh
inference_mobilenetv2_onnx.py               sample
inference_ssd_mobilenetv1_tflite.py         setup_venv_onnx.sh
input                                       setup_venv.sh
input_networks                           setup_venv_tflite.sh
labels                                  ssdlite300.anchor
my_dataset_path                          ssdlite300.onnx
parser_debug.log                         test_example
README.md                               test_fpga
requirements_onnx.txt                    tools
                                         tools_sdk
```

Yolov8 모델 변환 및 최적화

tc-nn-toolkit 환경 설정(WSL Ubuntu)

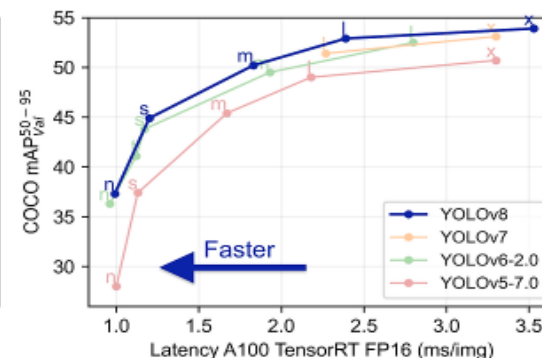
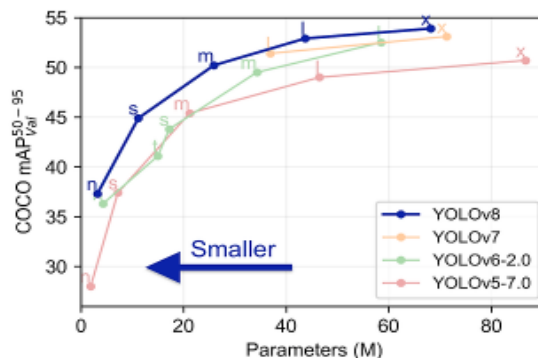
- 가상환경 세팅 & 필요 패키지 설치
 - `$ sudo add-apt-repository ppa:deadsnakes/ppa -y`
 - `$ sudo apt install python3.8 python3.8-venv python3.8-dev python3.8-tk`
 - `$ chmod 755 *`
 - `$ ./setup_venv.sh`
- 가상환경 활성화
 - `$ source venv/bin/activate`

Yolov8 모델 변환 및 최적화

tc-nn-toolkit 환경 설정(WSL Ubuntu)

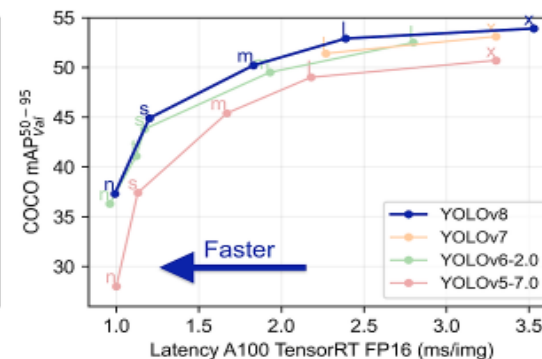
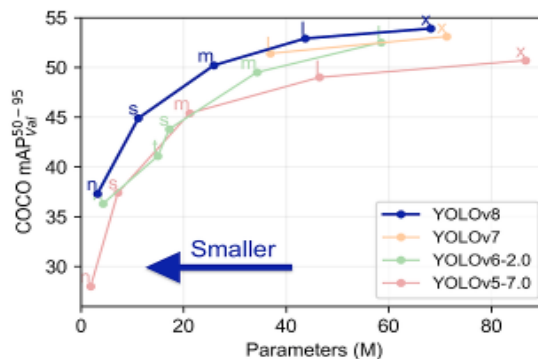
- `$ cd`
- `$ deactivate`
- `$ wget https://developer.arm.com/-/media/Files/downloads/gnu-a/9.2-2019.12/binrel/gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz`
- `$ tar -xvf gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu.tar.xz`
- `$ echo "export PATH=~/gcc-arm-9.2-2019.12-x86_64-aarch64-none-linux-gnu/bin:\$PATH" >> ~/.bashrc`
- `$ source ~/.bashrc`
- `$ aarch64-none-linux-gnu-gcc --version`

Yolov8 모델 변환 및 최적화



- 객체 감지, 이미지 분류, 인스턴스 Segmentation Task를 사용할 수 있는 통합 YOLO 모델
- Yolov8은 COCO와 Roboflow 100 데이터 셋에서 이전 모델 보다 더 높은 정확도를 보임
- 모델 크기 n/s/m/l/x로 사용하고자 하는 자원에 맞춰 속도 및 정확도 선택 가능
- ONNX/TFLite/TensorRT 등 배포 경로가 다양하여 임베디드 포팅 용이

Yolov8 모델 변환 및 최적화



- 주요 특징
 - Anchor-Free 구조 : 기존 YOLOv5 대비 경량화 + 빠른 학습 및 추론
 - Decoupled Head : 분류와 회귀를 분리하여 정확도 개선

Yolov8 모델 변환 및 최적화

- tc-nn-toolkit 가상 환경 세팅

```
$ cd tc-nn-toolkit
```

```
$ source venv/bin/activate
```

- Input 디렉터리에 yolov8s 모델 가져오기

```
$ cd input_networks
```

```
$ git clone https://github.com/topst-development/Education
```

```
$ cd Education
```

```
$ git switch edu/3_day_course
```

```
$ ls ai_model/
```

```
ead@TC-D-20-0546:~/test_lib/Education/ai_model$ ls  
yolov8s.bin  yolov8s_extracted.onnx
```

```
$ cd ~/tc-nn-toolkit
```

Yolov8 모델 변환 및 최적화

- 모델 컨버팅 수행

```
$ python ./EnlightSDK/converter.py ./input_networks/Education/ai_model/yolov8s_extracted.onnx  
--type obj \  
--add-detection-post-process ./input_networks/Education/ai_model/yolov8s.bin \  
--dataset Custom \  
--dataset-root my_dataset_path \  
--output ./output_networks/yolov8s.enlight \  
--enable-track \  
--mean 0 0 0 \  
--std 1 1 1 \  
--num-class 80 \  
--yolo-version v8 \  
--enable-letterbox \  
--dfl-reg-max 16 \  
--output-order cl --num-images 1
```

Yolov8 모델 변환 및 최적화

- 모델 컨버팅 수행

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit$ python ./EnlightSDK/converter.py \
./input_networks/Education/ai_model/yolov8s_extracted.onnx \
--type obj \
--add-detection-post-process ./input_networks/Education/ai_model/yolov8s.bin \
--dataset Custom \
--dataset-root my_dataset_path \
--output ./output_networks/yolov8s.enlight \
--enable-track \
--mean 0 0 0 \
--std 1 1 1 \
--num-class 80 \
--yolo-version v8 \
--enable-letterbox \
--dfl-reg-max 16 \
--output-order cl
```

- type : 네트워크 모델의 후처리 타입 결정 (obj, cls, unknown)
- add-detection-post-process : yolo 모델의 앵커 박스 적용 옵션
- dataset : 양자화 스케일 수집을 위한 데이터셋 타입
- dataset : 양자화 스케일 수집에 사용할 입력 데이터 경로
- output : 모델 변환 결과 저장 위치
- mean : 입력 데이터 정규화 파라미터 (채널별 평균값)

Yolov8 모델 변환 및 최적화

- 모델 컨버팅 수행

```
(venv) ead@TC-D-20-0546:~/tc-nn-toolkit/tc-nn-toolkit$ python ./EnlightSDK/converter.py \
./input_networks/Education/ai_model/yolov8s_extracted.onnx \
--type obj \
--add-detection-post-process ./input_networks/Education/ai_model/yolov8s.bin \
--dataset Custom \
--dataset-root my_dataset_path \
--output ./output_networks/yolov8s.enlight \
--enable-track \
--mean 0 0 0 \
--std 1 1 1 \
--num-class 80 \
--yolo-version v8 \
--enable-letterbox \
--dfl-reg-max 16 \
--output-order cl
```

- std : 입력 데이터 정규화 파라미터 (채널 별 표준편차)
- num-class : 모델 class 수
- yolo-version : yolo 모델의 버전 설정
- enable-letterbox : letterbox 옵션 설정 여부
- dfl-reg-max : yolo의 dfl 레이어 좌표 분포 수
- output-order : 모델 출력 텐서 순서 -> cl : [N, H, W, C]

Yolov8 모델 변환 및 최적화

- 모델 양자화 수행

```
$ python ./EnlightSDK/quantizer.py ./output_networks/yolov8s.enlight --  
output ./output_networks/yolov8s_quantized.enlight
```

```
Start FuseConstantEltwLayer optimizer  
debugizing 127 / 219  
optimizing ..... done.  
Start TreatIntermediateOutput optimizer  
debugizing 127 / 219  
optimizing ..... done.  
Start ModifyLayerQuantizationFriendly optimizer  
debugizing 142 / 250  
optimizing ..... done.  
quantization..... done.  
End sanity check for custom quantization  
Start MakeCompilerFriendly optimizer  
debugizing 127 / 252  
optimizing ..... done.  
Start MakeLayerAligned optimizer  
debugizing 127 / 243  
optimizing ..... done.  
Serializing Graph done.  
Writing to File..... Done  
Writing File path : ./output_networks/yolov8s_quantized.enlight  
  
Quantizer done.
```

Yolov8 모델 변환 및 최적화

- 모델 컴파일 수행

```
$ python ./EnlightSDK/compiler.py ./output_networks/yolov8s_quantized.enlight --th-iou 0.5 --th-conf 0.5
```

```
Start MakeCode(version=NPUV20) code-emitting
code-emitting ..... done.
                ./output_code/yolov8s_quantized/npu_cmd.h created
                ./output_code/yolov8s_quantized/npu_cmd.bin created
                ./output_code/yolov8s_quantized/npu_cmd.txt created
Writing [11196416]/[11196416]
                ./output_code/yolov8s_quantized/quantized_network.bin created
                ./output_code/yolov8s_quantized/oact_entry_table.py created
Notification!!!
Conv2d_42___model_22_cv3_0_cv3_0_2_Conv, Conv2d_53___model_22_cv3_1_cv3_1_2_Conv, Conv2d_62___model_22_cv3_2_cv3_2_2_Conv assigned to conf layers
Conv2d_41___model_22_cv2_0_cv2_0_2_Conv, Conv2d_52___model_22_cv2_1_cv2_1_2_Conv, Conv2d_61___model_22_cv2_2_cv2_2_2_Conv assigned to loc layers
                ./output_code/yolov8s_quantized/network.h created
                ./output_code/yolov8s_quantized/post_process.c created
                ./output_code/yolov8s_quantized/oe_network.bin created
                ./output_code/yolov8s_quantized/oe_network.h created

                NPU compiler: done sucessfully.

Compiler. done.
```


Yolov8 모델 변환 및 최적화

- 네트워크 및 후처리 빌드 수행

- 모델 복사

- \$ cd build_network

- \$ cp ../output_code/yolov8s_quantized/ ./ -ar

- 링크된 파일 제거

- \$ rm -rf network.h post_process.c

- Yolov8s 네트워크 및 후처리 파일 링크

- \$ ln -s yolov8s_quantized/network.h

- \$ ln -s yolov8s_quantized/post_process.c

- 후처리 빌드 수행

- \$ make

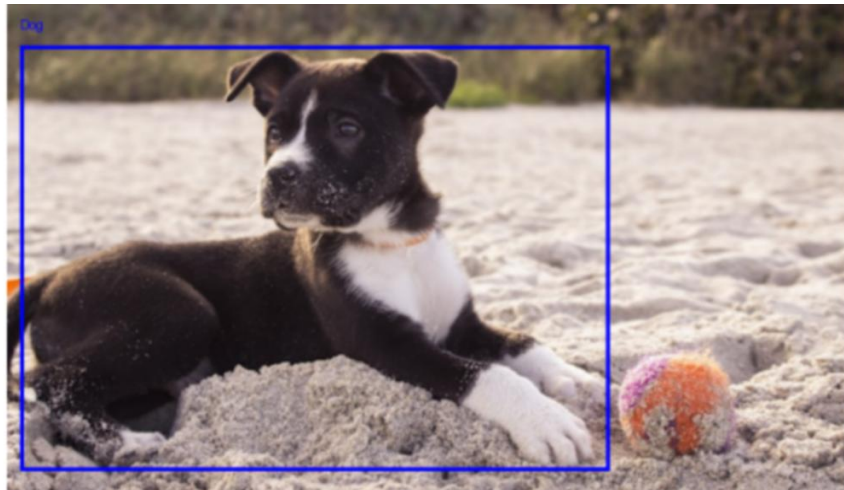
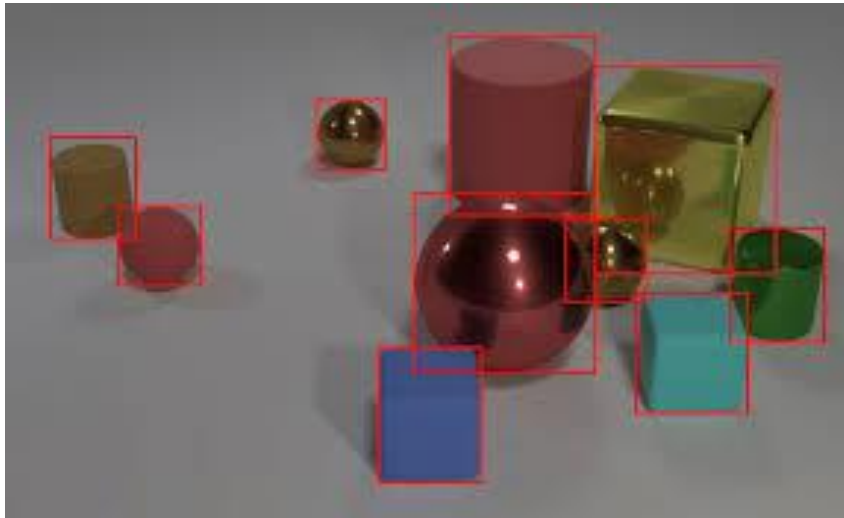
```
aarch64-none-linux-gnu-gcc -shared -Wl,-soname,net.so -o net.so post_process.o detector.o classifier.o yolo_detector.o ssd_detector.o custom_postproc.o enlight_network.o network.o  
rm -rf ./*.o network/*.o enlight/*.o /*.o
```

Yolov8 모델 변환 및 최적화

- 네트워크 및 후처리 빌드 수행

- onnx 모델의 경우 모델 내부에서 후처리를 수행하지 않기 때문에 특정 텐서 형태로 출력됨
- 해당 출력 텐서를 api 호출을 통해 불러와 추가적인 연산을 진행하여 객체인식을 위한 바운딩 박스 처리를 진행해야함
- Yolo의 경우 다음과 같은 연산을 수행
 - DFL 연산 (softmax + Expectation)
 - Bounding Box 복원 (Decoding)
 - Sigmoid 연산
 - Confidence 계산
 - Threshold Filtering
 - Non-Maximum Suppression (NMS)
 - Output 출력 (bbox 좌표, score, class, label)

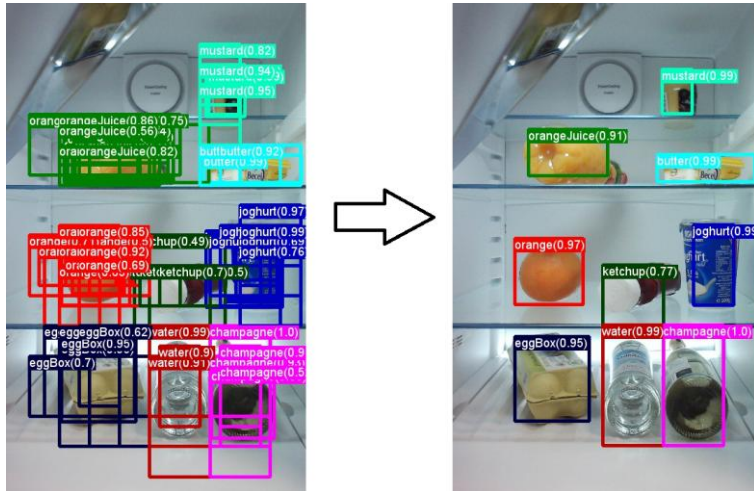
Yolov8 모델 변환 및 최적화



- Bounding Box

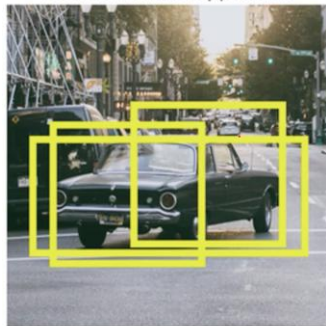
- 이미지 내에서 객체의 위치를 나타내는 직사각형 영역
- Yolo모델은 객체의 위치와 크기를 예측
- 구성 요소
 - x, y 좌표 (중앙점)
 - w, h 값 (너비, 높이)

Yolov8 모델 변환 및 최적화

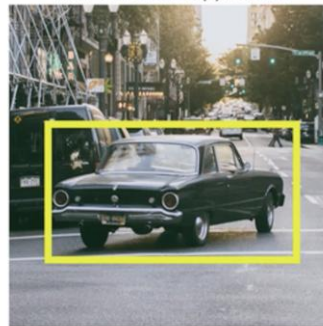


Before non-max suppression

After non-max suppression

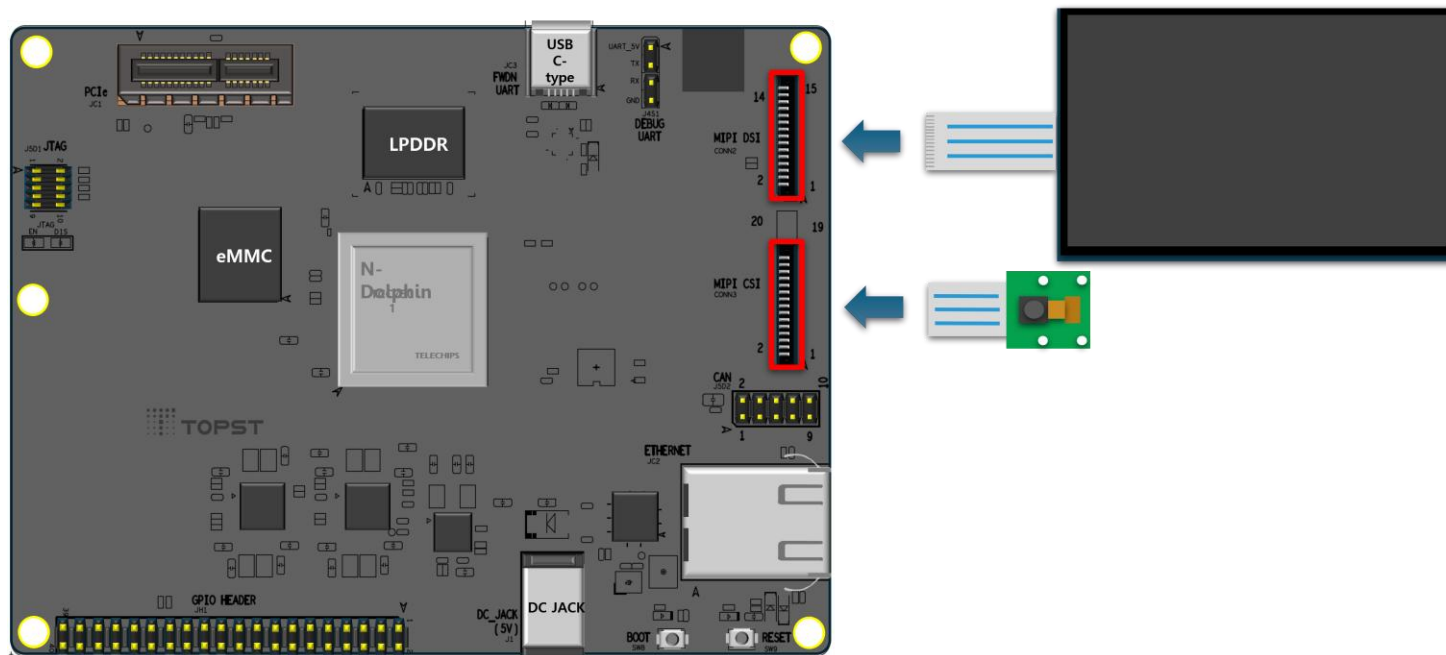


Non-Max
Suppression



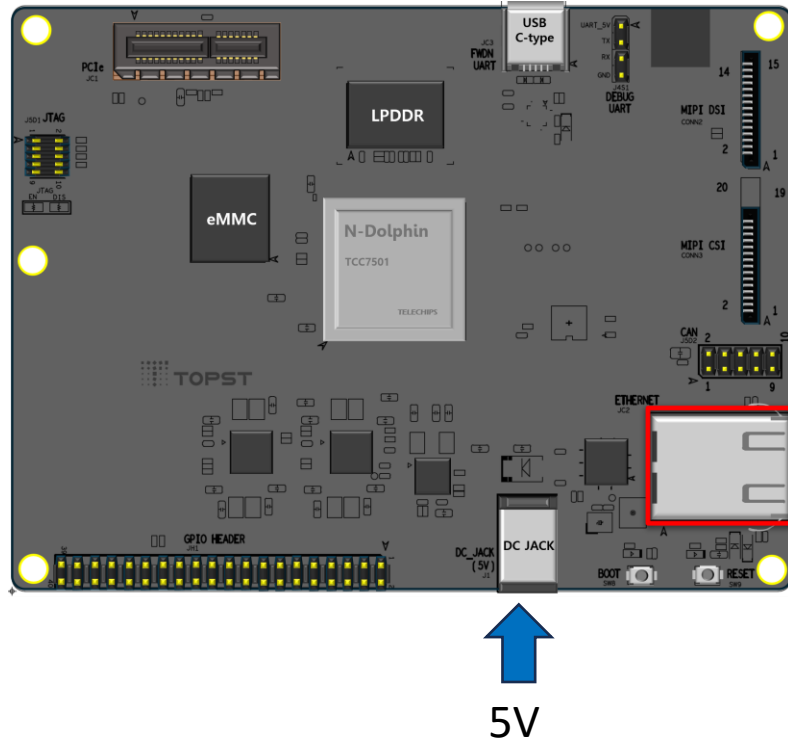
- NMS - 중복 탐지 제거
 - 여러 개의 후보 박스 중 최고 점수를 가진 박스를 선택
 - 중복된 박스는 제거
 - 설정된 임계값 이상인 박스들은 중복으로 간주

Yolov8 모델 변환 및 최적화



- 모델 추론의 입/출력을 위한 카메라 및 디스플레이 연결

Yolov8 모델 변환 및 최적화

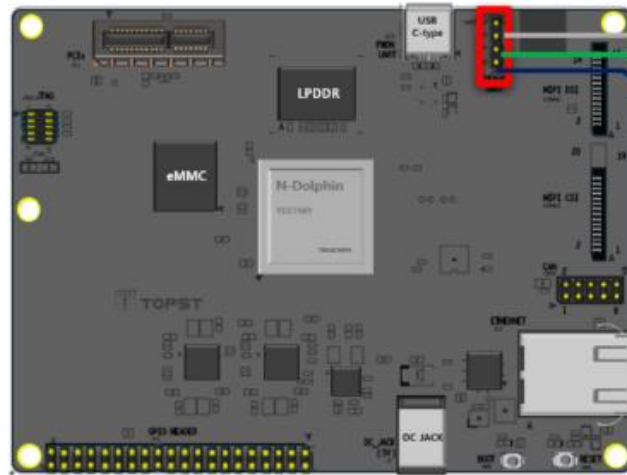


Ethernet Cable



- 이더넷 및 5V 전원 연결

Yolov8 모델 변환 및 최적화

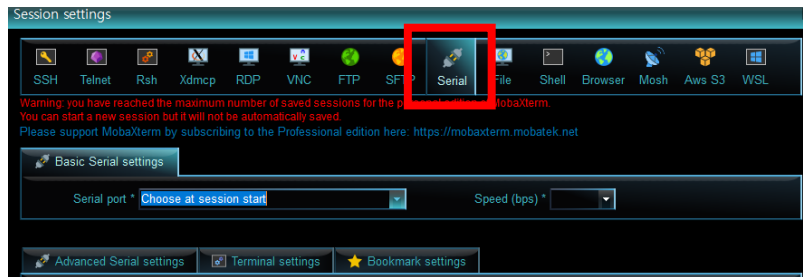
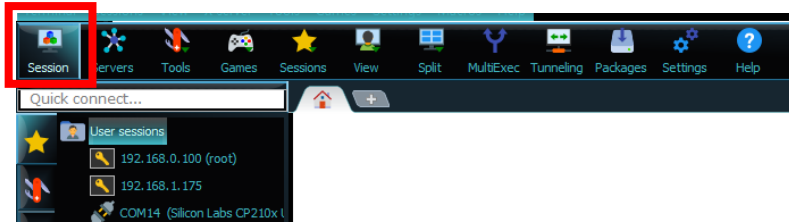


UART to TTL Cable

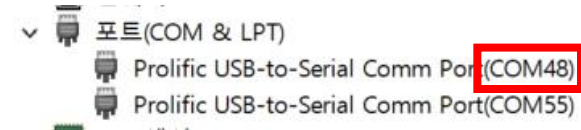


- UART 디버거 연결

Yolov8 모델 변환 및 최적화



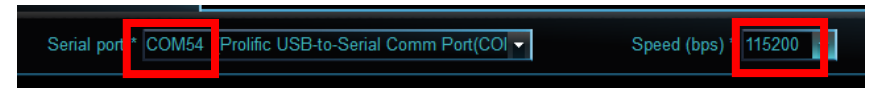
- 윈도우 검색창에 MobaXterm 검색 후 실행
- 오른쪽 상단에 Session 버튼 클릭
- Serial 클릭
- 장치 관리자를 통해 port 확인



Yolov8 모델 변환 및 최적화



- Serial port에서 연결된 포트 설정
- Speed 115200 설정



- Advanced Serial settings 탭 선택
- Flow contro에서 None 선택
- OK 눌러 보드 접속
- ID : root / PW : root 입력

Yolov8 모델 변환 및 최적화

- 네트워크 및 후처리 빌드 수행

- 후처리 파일 복사 및 AI-G 보드 전송 (wsl ubuntu에서 실행)

- \$ cp net.so yolov8s_quantized

- \$ scp -r yolov8s_quantized root@192.168.0.100:/home/root/ -> 비밀번호 root 입력

- 이후 실습에 사용할 추론 앱 전송

- \$ scp -r ../input_networks/Education/ai-g\ app/tcnnapp root@192.168.0.100:/home/root/

Yolov8 모델 변환 및 최적화

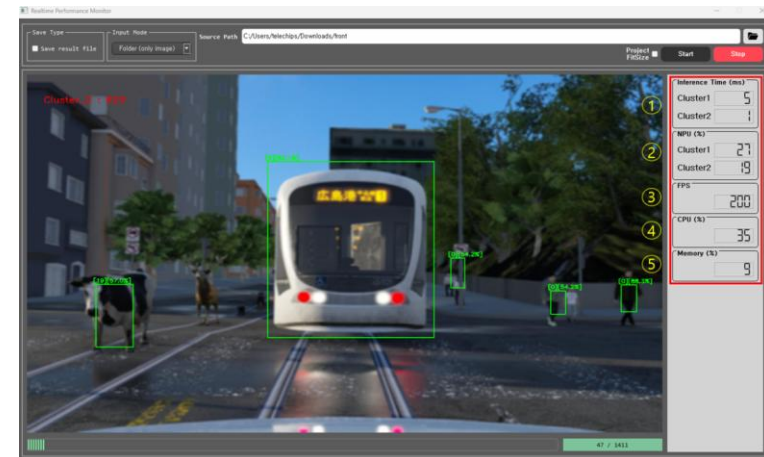
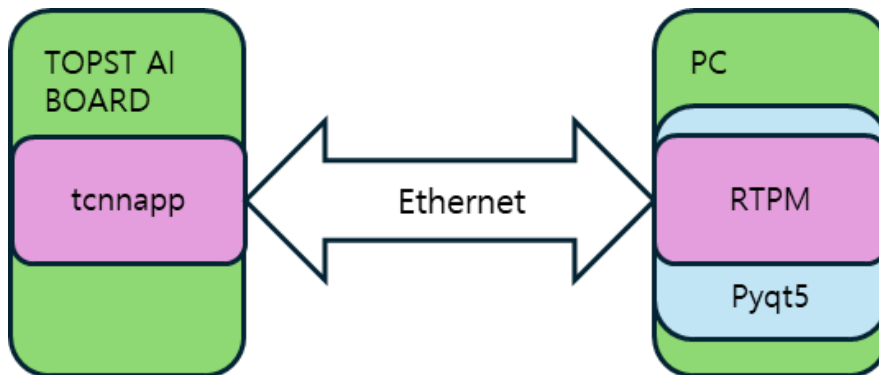
- NPU 추론 수행
 - CSI CAMERA, DSI DISPLAY 연결 후 보드 부팅
 - 추론 명령어 입력

```
$ tcnnapp -n yolov8s_quantized -p /dev/video2
```
 - 추론 결과 확인



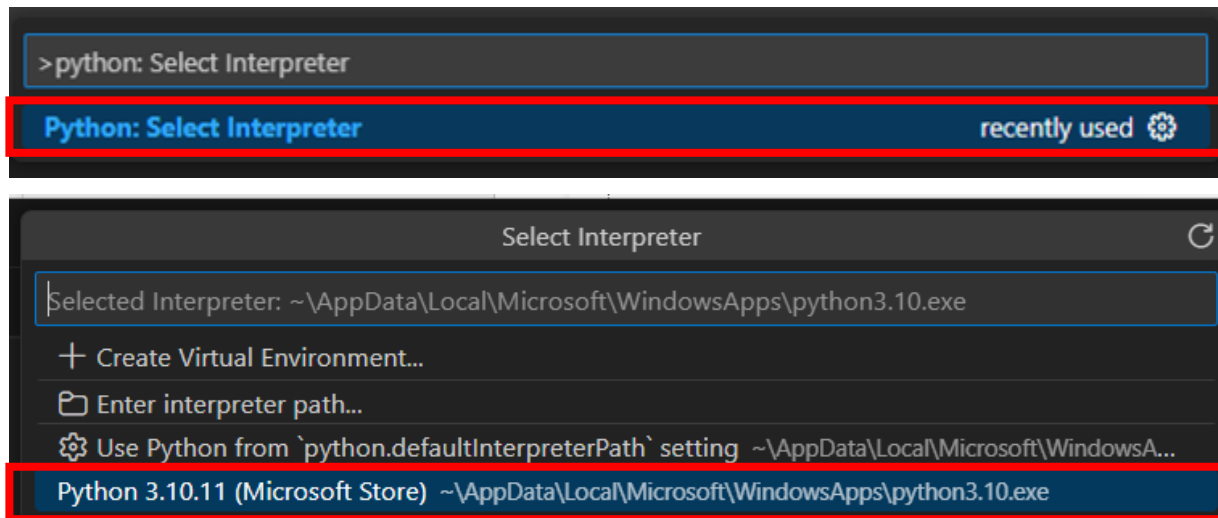
RTPM 기반 모델 추론 실습

- RTPM(Real-Time Performance Monitor)
 - 파이썬 기반 실시간 성능 모니터링 툴킷
 - AI-G 추론 과정에서 얻은 이미지, NPU, CPU 및 MEM 성능을 PC로 전송
 - 결과를 PC의 RTPM 툴로 실시간 모니터링 및 분석
 - RTPM Download : <[RTPM](#)>



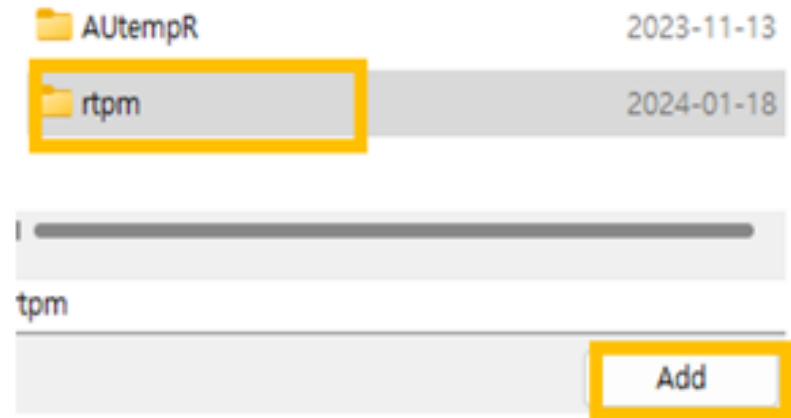
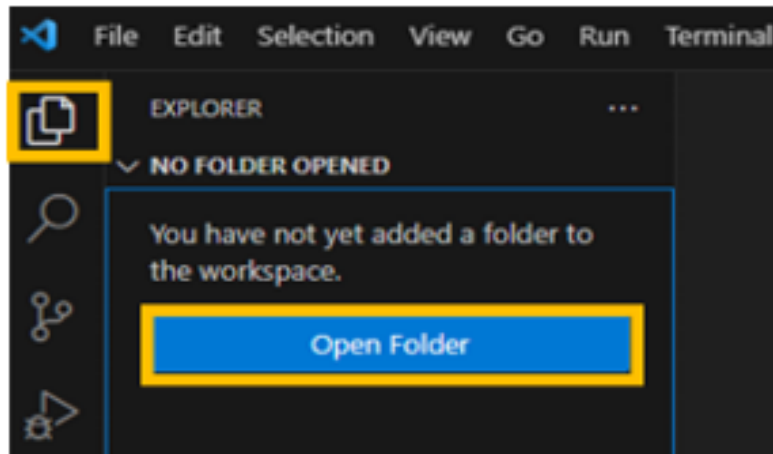
RTPM 기반 모델 추론 실습

- RTPM Tool 실행
 - 파일 탐색기에 visual studio code 검색 후 실행
 - Ctrl + Shift + P를 입력
 - **Python: Select Interpreter** 선택
 - **Python 3.10.xx** 선택



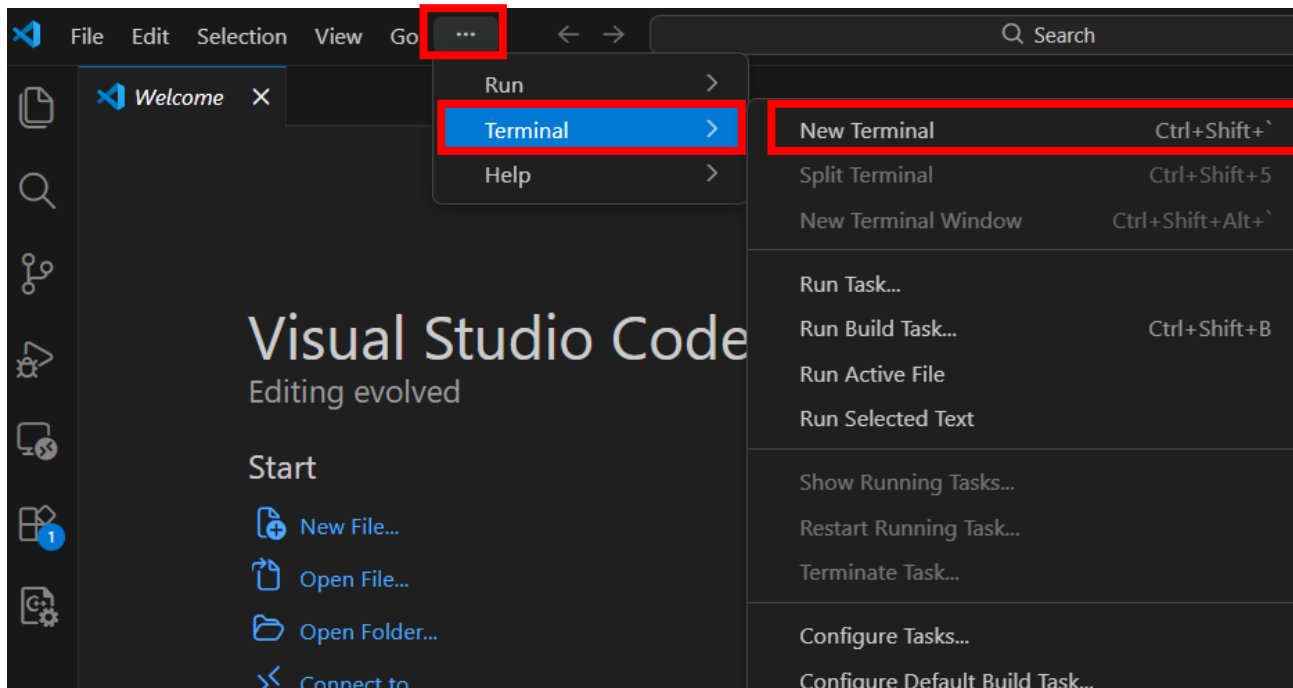
RTPM 기반 모델 추론 실습

- RTPM Tool 실행
 - Vscode의 왼쪽 상단에 explorer 아이콘 선택
 - 사전 다운로드 된 rtpm 폴더 열기



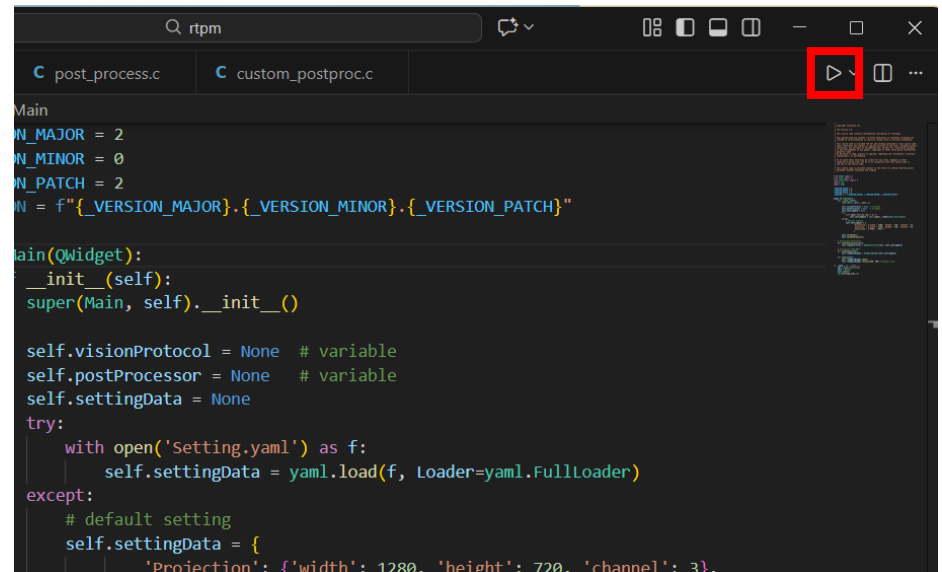
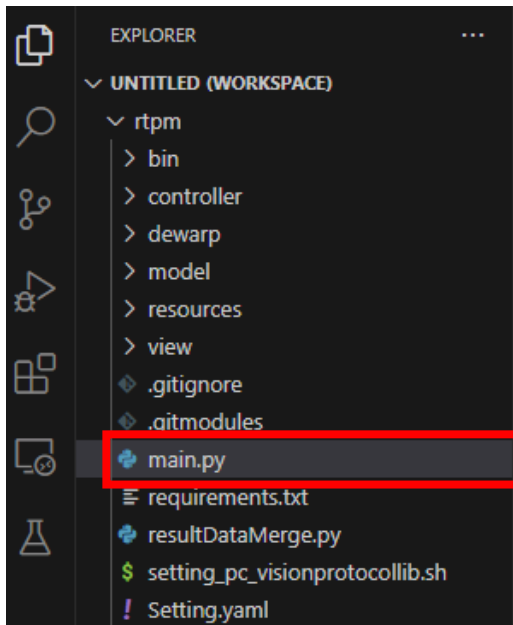
RTPM 기반 모델 추론 실습

- RTPM Tool 개발 환경 설정
 - ... -> Terminal -> New Terminal 선택
 - 터미널 창에 해당 명령어 입력 "**pip install -r requirements.txt**"



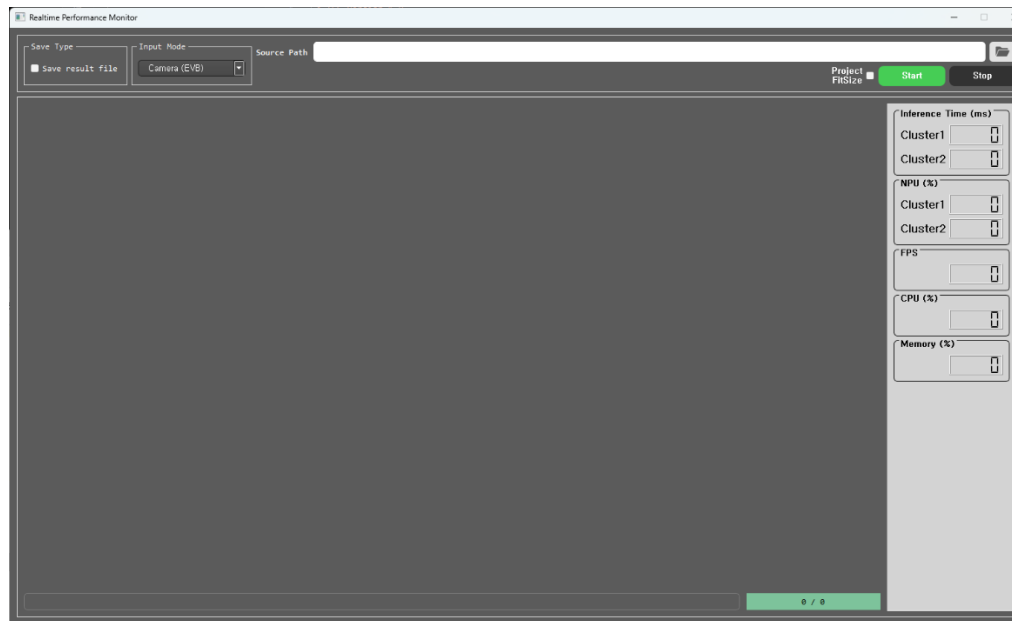
RTPM 기반 모델 추론 실습

- RTPM Tool 실행
 - RTPM 폴더에서 main.py 선택
 - 오른쪽 상단 시작 버튼으로 main.py 코드 실행



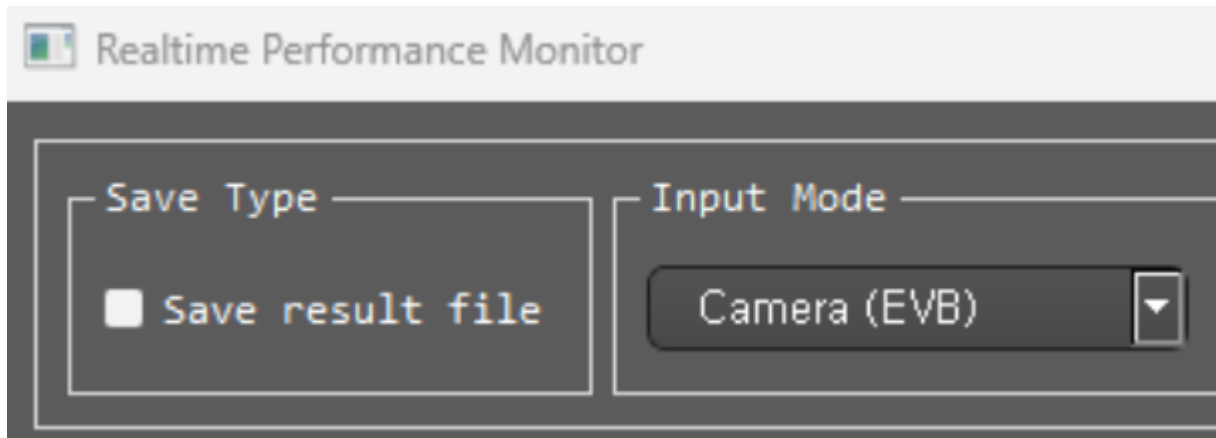
RTPM 기반 모델 추론 실습

- RTPM Tool 실행
 - RTPM 폴더에서 main.py 선택
 - 오른쪽 상단 시작 버튼으로 main.py 코드 실행



RTPM 기반 모델 추론 실습

- RTPM Tool 실행
 - 카메라 모드로 실행하기 위해 옵션 선택
 - Input Mode를 Camera(EVB)로 설정
 - Start 버튼 선택

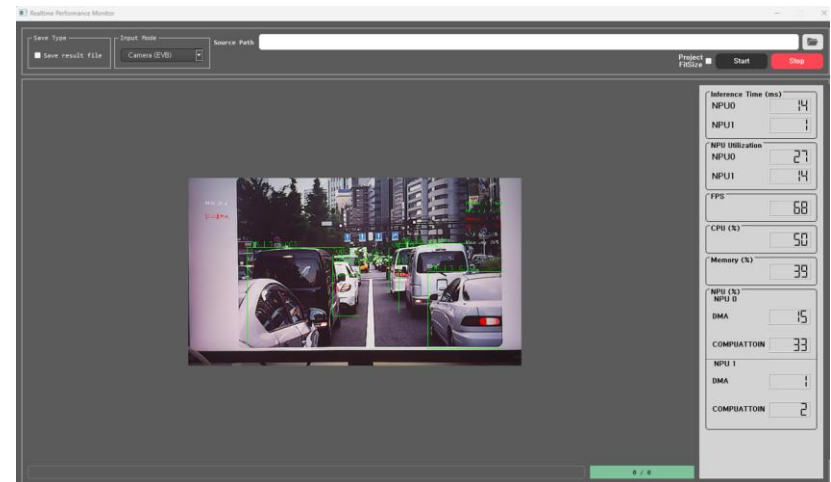


- RTPM 기반 AI-G 추론 실행

- AI-G 터미널 창 접속
- Host PC와 Board 이더넷 접속 확인
- RTPM 기반 추론 명령어 실행
- `$ tcnnapp -n yolov8s_quantized -p /dev/video2 -i camera -o rtpm`

[illegible]

AI-G 터미널 화면



RTPM 화면

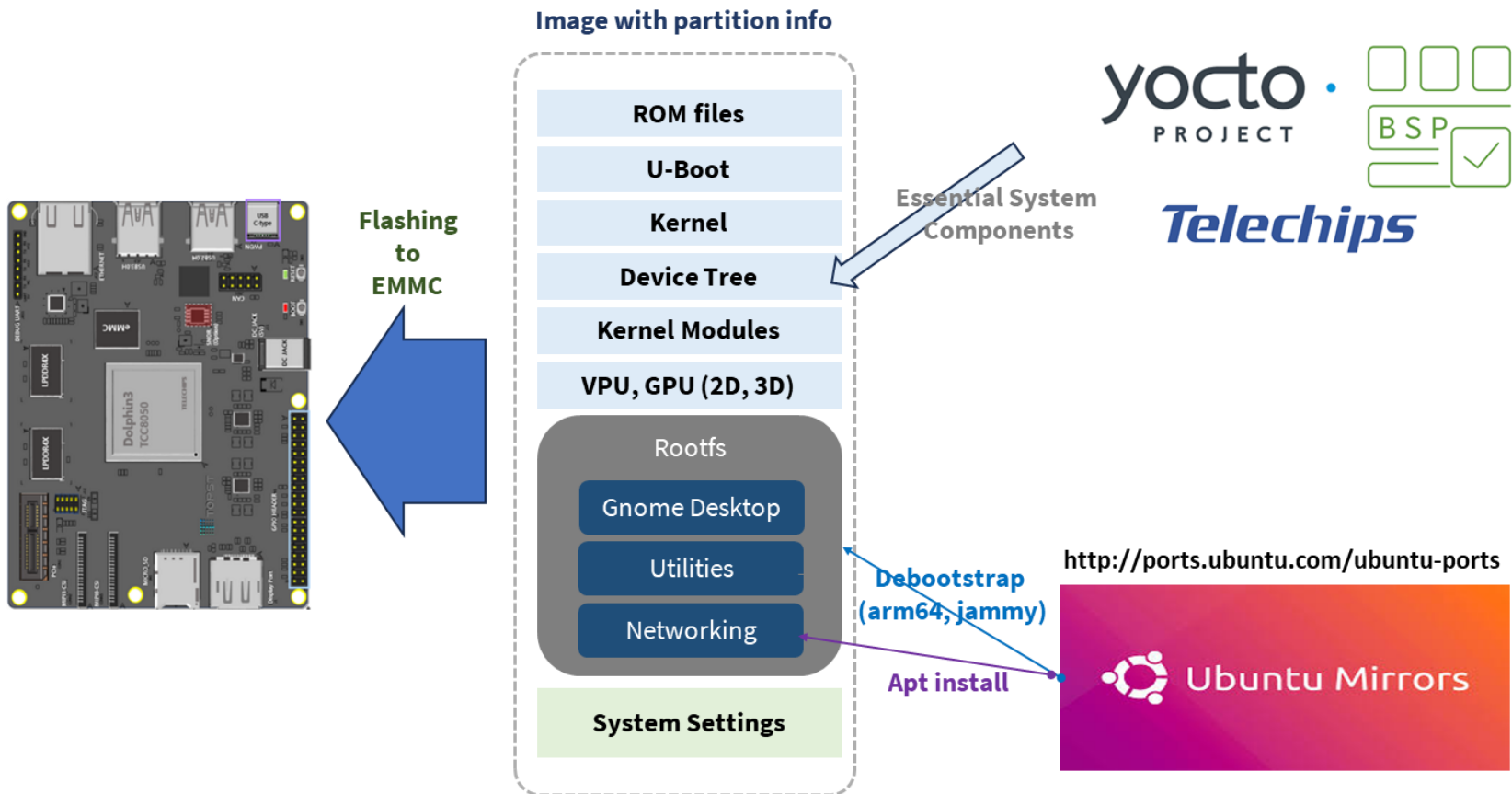
D3-G 개발 환경 설정

Part Name	D3-G
CPU	Arm® Cortex-A72 1.6 GHz, Quad
CPU	Arm® Cortex-A53 1.45 GHz, Quad
CPU	Arm® Cortex-R5
GPU	168 GFLOPS, IMG PowerVR 9XTP
Memory	4 GB (2 GB 32-bit LPDDR4 x 2)
Storage	microSD Card (Not included), 8 GB eMMC 5.1 Flash Storage
Automotive Network	CAN 3-channel
Network	Gigabit Ethernet
Camera	MIPI CSI-2 2-lane, MIPI CSI-2 4-lane
Display	DisplayPort 1.4
USB	USB 2.0 Host, USB 2.0 DRD, USB 3.0 DRD
Others	PCIe Gen3, UART/JTAG debugger port, I2S, I2C, SPI, PWM
Power	5V/3A
OS	Linux
Mechanical	120 mm x 90 mm

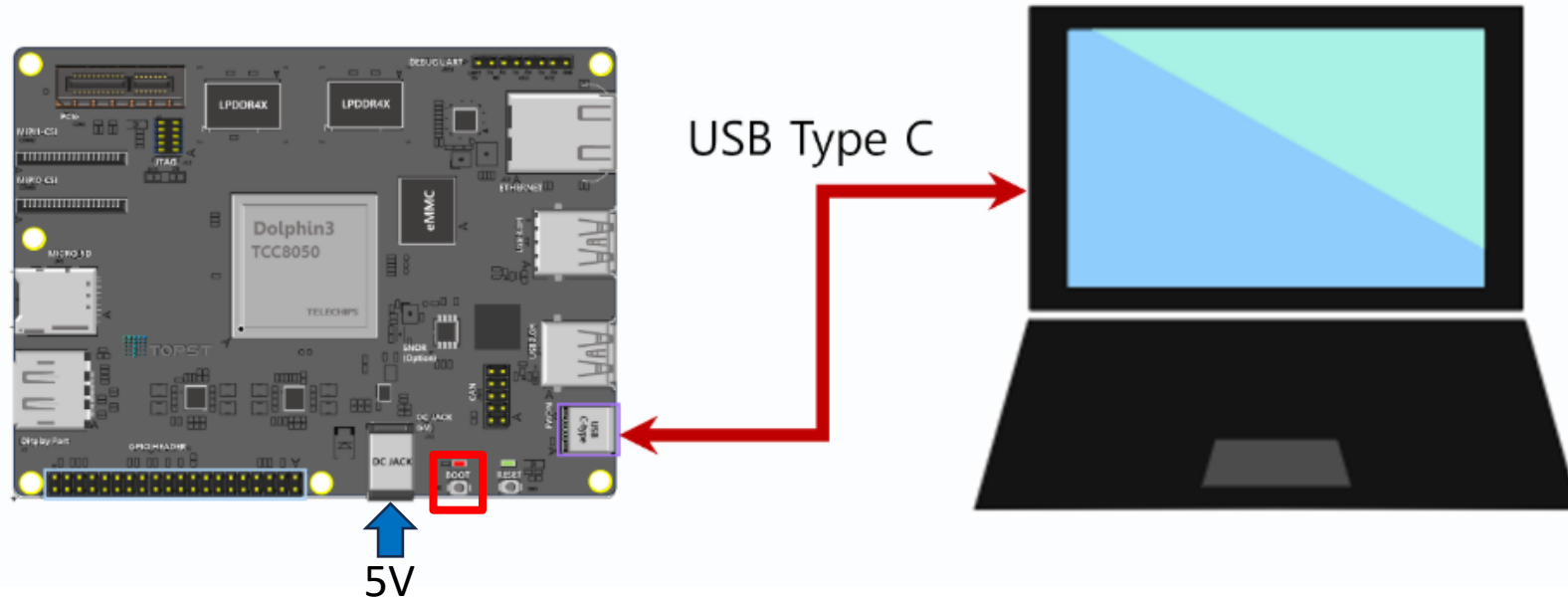
D3-G Specification

- D3-G 는 3개의 코어 존재
 - Cortex-A72
 - Cortex-A53
 - Cortex-R5
- 본 실습에선 **A72 코어와 R5 코어만**을 사용
 - A72 – Ubuntu Weston Desktop 설치
 - R5 – FreeRTOS 설치

D3-G 개발 환경 설정



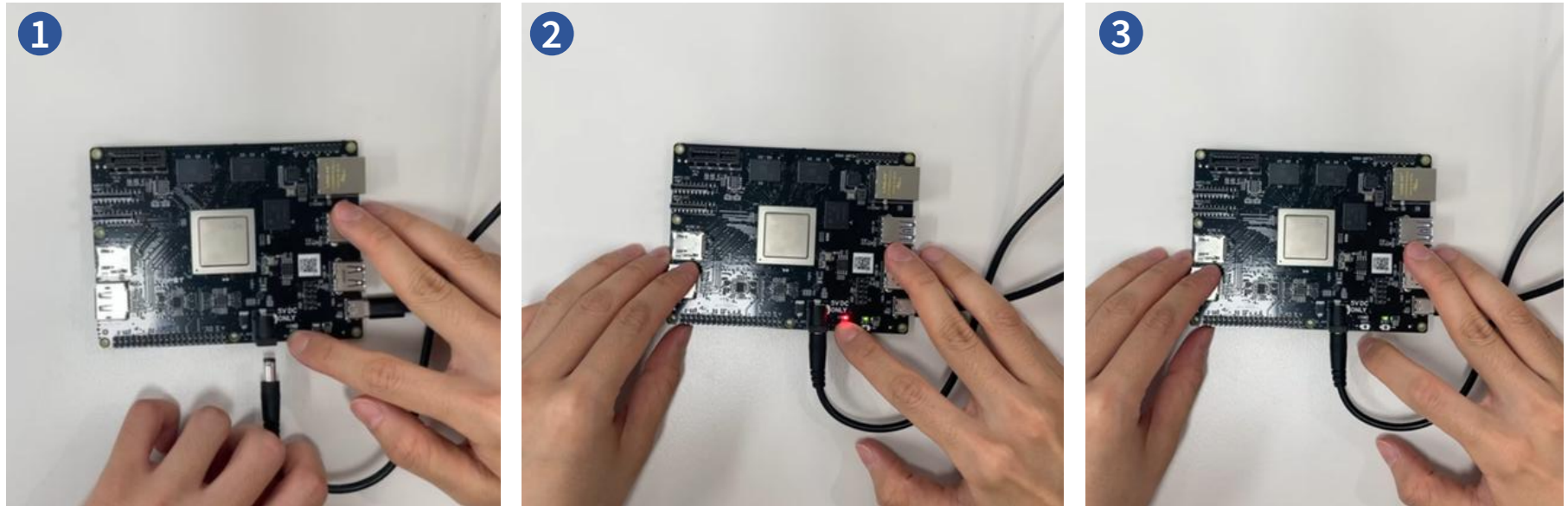
D3-G 개발 환경 설정



- **Connect D3-G Board and Host PC with USB Boot Mode**

- USB Boot Mode로 들어가기 위해 **FWDN** 스위치를 누른 상태에서 전원 케이블(5V)을 D3-G 보드에 연결
- **USB Type-C** 케이블을 D3-G 보드 및 Host PC에 연결
- D3-G : **C** 케이블 & Host PC : **USB** 케이블

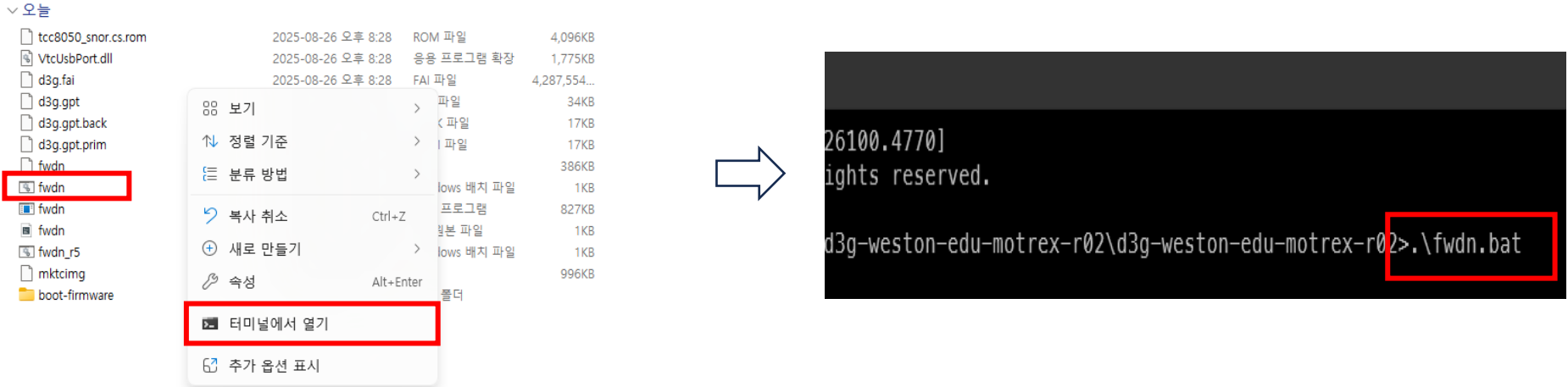
D3-G 개발 환경 설정



- **Boot Mode 진입**

- BOOT 스위치를 누른 채로, 전원 플러그 연결(BOOT Switch LED 상태 : ON)
- 이후 스위치에서 손가락을 떼면 진입 완료(BOOT Switch LED 상태 : OFF)

D3-G 개발 환경 설정

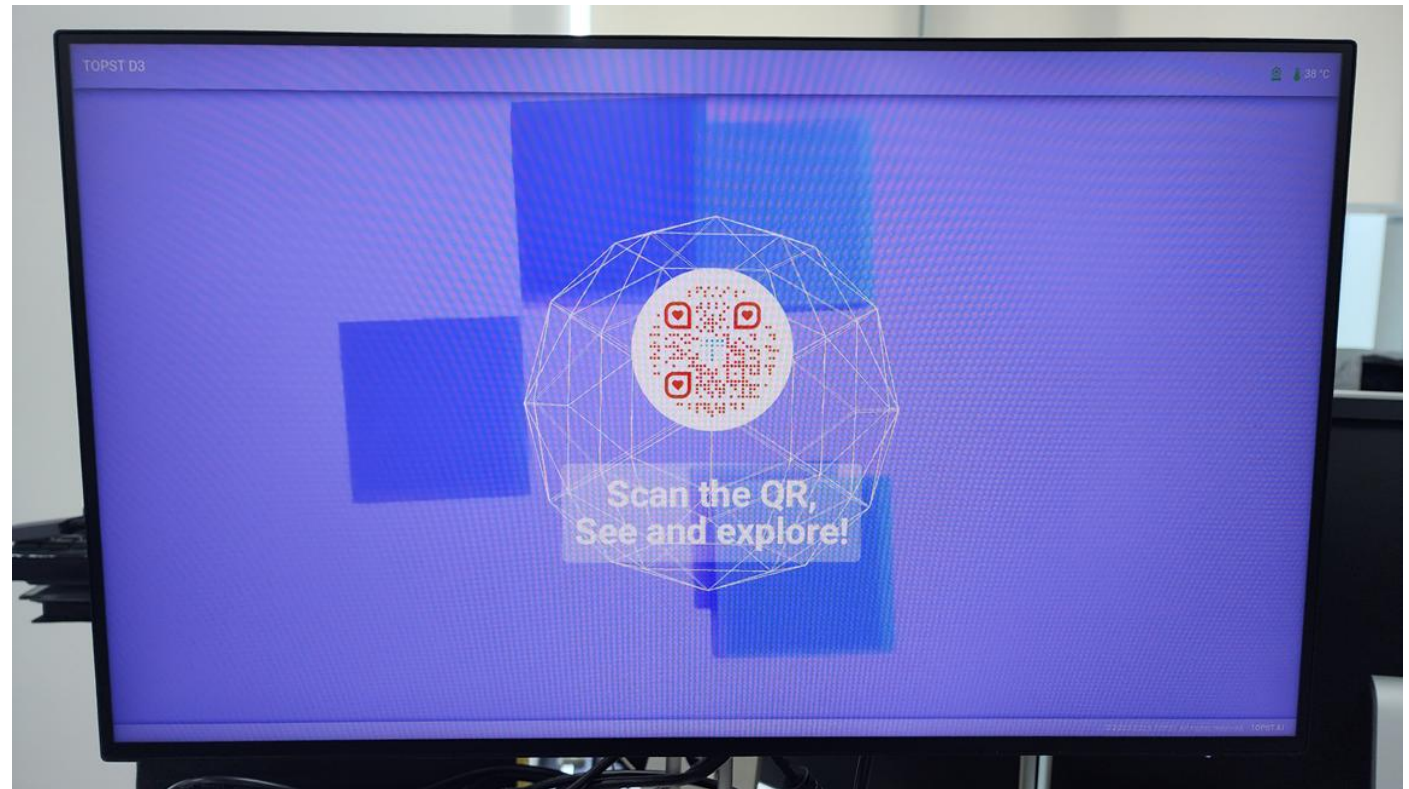


• FWDN (D3-G Ubuntu Weston/Gnome Desktop)

- 제공된 3_Day_course.zip 파일을 PC Windows 혹은 PC ubuntu에 다운로드 후 압축 해제
- Ex) Windows
 - 받은 폴더 내에서 마우스 우클릭> 터미널에서 열기(명령프롬프트) 실행
 - `fwdn.bat` (Windows 배치 파일)실행
 - `fwdn.bat`은 펌웨어를 자동으로 다운로드 하는 실행 파일
 - `$ ./fwdn.bat`

D3-G 개발 환경 설정

- 설치가 완료되면 전원 재인가 or reset 버튼 누르기



Demo Web-site on localhost

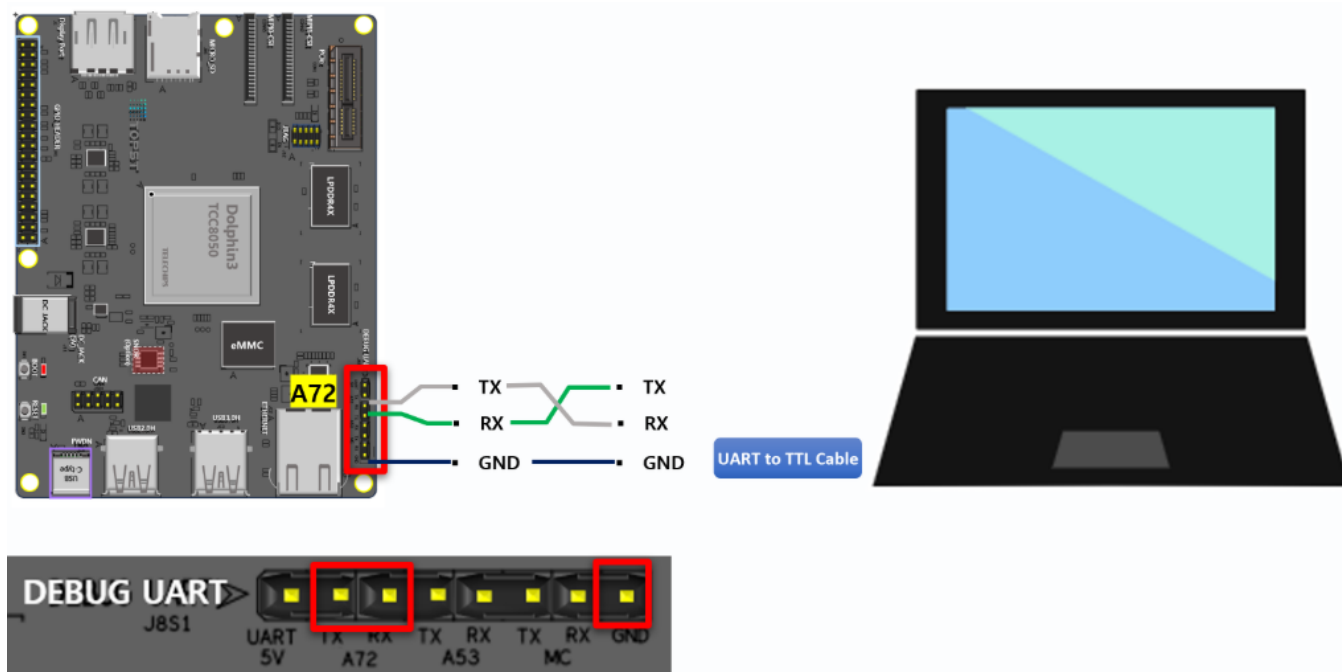
COG browser

WPE Webkit

WPE Backend

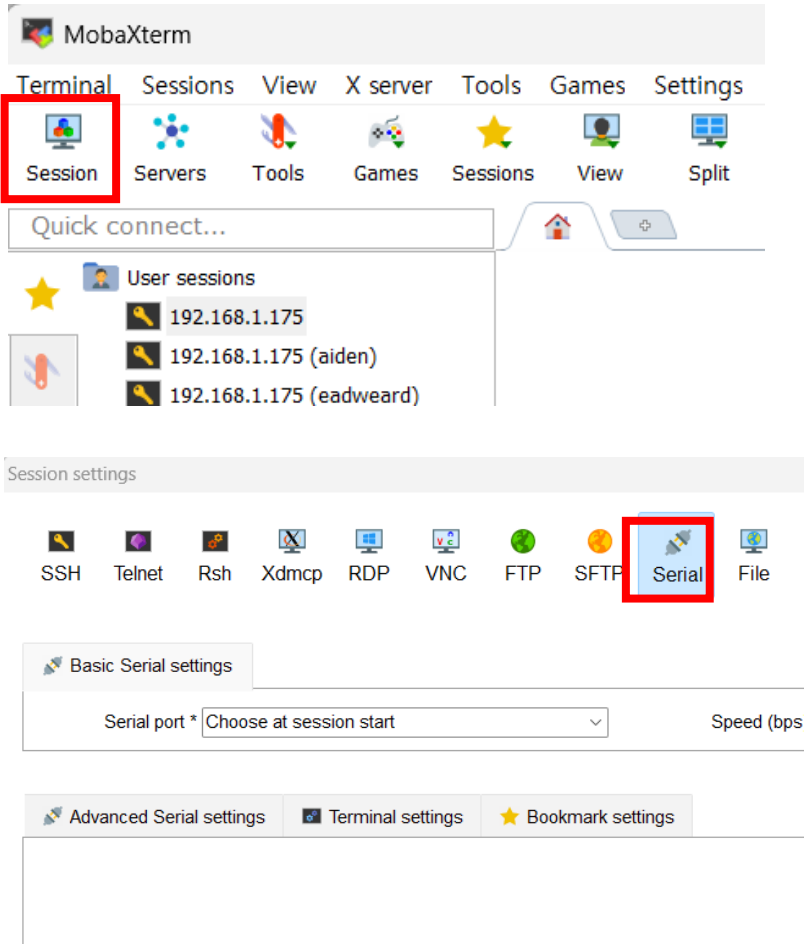
D3-G system

D3-G 개발 환경 설정

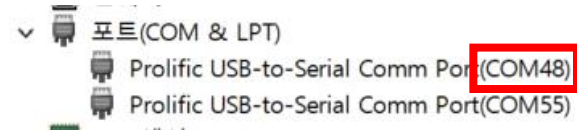


- **Connect D3-G A72 and Host PC with UART**
 - D3-G A72 코어와 PC 간 USB-to-TTL cable을 이용하여 연결
 - 검정 케이블은 UART 핀의 **GND**에 연결
 - 흰 케이블은 UART 핀의 **TX** 핀에 연결 & 녹색 케이블은 UART 핀의 **RX** 핀에 연결

D3-G 개발 환경 설정



- 윈도우 검색창에 MobaXterm 검색 후 실행
- 오른쪽 상단에 Session 버튼 클릭
- Serial 클릭
- 장치 관리자를 통해 port 확인



D3-G 개발 환경 설정

Basic Serial settings

Serial port * COM4 (Prolific USB-to-Serial Comm Port(CO... Speed (bps) * 115200

Advanced Serial settings Terminal settings Bookmark settings

Serial engine: PuTTY (allows manual COM port setting)

Data bits 8

Stop bits 1

Parity None

Flow control None

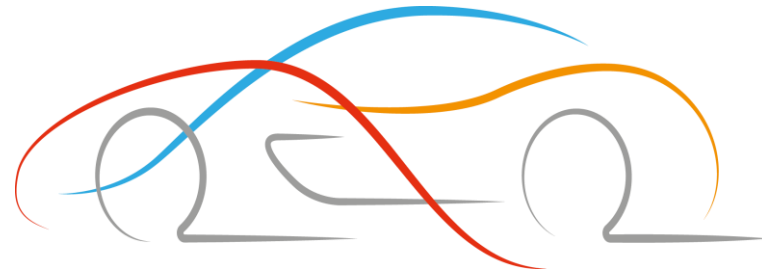
Reset defaults

Execute macro at session start: <none>

If you need to transfer files (e.g. router configuration file), you can use MobaXterm embedded TFTP server

"Servers" window --> TFTP server

- Serial port에서 연결된 포트 설정
 - Speed 115200 설정
- Advanced Serial settings 탭 선택
 - Flow control에서 None 선택
- ok눌러 보드 접속
- ID : topst / PW : topst 입력



Thank You